

Distributed and Pervasive Systems

simi

February 2024

Contents

1	Introduction	5
1.1	Definitions	6
1.2	Goals of a distributed system	7
1.2.1	Transparency	7
1.3	Openness	8
1.4	Scalability	8
1.5	Scaling technique	9
1.6	False assumptions	10
1.7	Types of Distributed System	10
1.7.1	Clusters Computing Systems	11
1.7.2	Kubernetes	12
1.7.3	Cloud Computing	12
1.7.4	Edge Computing	13
1.8	Pervasive computing	13
2	Architecture	15
2.1	Centralized Architectures	16
2.1.1	Vertical distribution	17
2.1.2	Horizontal distribution	18
2.1.3	Vertical and horizontal combined	18
2.1.4	Microservice architecture	19
2.1.5	Container	19
2.2	Event Bus Architecture	20
2.3	Decentralized architectures	21
2.3.1	Middleware	22
2.3.2	Hybrid architectures in Collaborative Distributed Systems	25
3	Communication	27
3.1	Types of communication	28
3.2	Message Oriented Transient	31
3.2.1	Queuing Systems	32
3.3	Remote Procedure Call	34
3.3.1	Conventional Procedure Call	34
3.3.2	Client and Server Stubs	35
3.3.3	Marshalling and Unmarshaling	36
3.3.4	Writing client and server	38

3.3.5	Binding a Client to Server	38
4	Synchronization problems in distributed system	39
4.1	Synchronize physical clocks	41
4.1.1	GNSS	41
4.1.2	Network Time Protocol	42
4.1.3	The Berkeley Algorithm	43
4.2	Lamport's Logical Clocks	43
4.2.1	Totally ordered Multicasting	46
4.3	Mutual exclusion	46
4.3.1	Centralized algorithm	46
4.3.2	Ricart and Agrawala	47
4.3.3	Ring algorithm	48
4.4	Election algorithms	49
4.4.1	Bully	49
4.4.2	Ring-based election	50
5	Fault tolerance	52
5.1	Failure Masking by Redundancy	54
5.1.1	Process resilience	54
5.1.2	How much redundancy?	54
5.2	Agreement problems	54
5.2.1	Synchronous versus asynchronous systems	55
5.2.2	Result on Byzantine agreement with synchronous systems	55
5.2.3	The CAP theorem for asynchronous systems	56
5.3	Paxos protocol	56
5.4	Raft protocol	57
5.4.1	Leader Election	57
5.4.2	Log Replication	58
5.4.3	TimeOuts	59
5.4.4	Network Partition	62
5.5	Other important topics	65
6	Distributed Ledger Technologies and Blockchain	66
6.1	Web3	67
6.2	The DLT System Model	67
6.3	Blockchain	68
6.3.1	Data in Blockchain	69
6.3.2	Approach	69
6.3.3	Problems	70
6.3.4	Consensus in blockchain	71
6.3.5	Hashing	71
6.3.6	Examples	72
6.3.7	Proof of Work algorithm	73
6.3.8	Properties	73
6.3.9	PoW limitation	74
6.3.10	Different types of consensus algorithms	74

6.4	Smart Contracts	75
6.5	Bitcoin and Ethereum are permissionless	75
6.5.1	Hyperledger Fabric	76
7	Pervasive Computing	77
7.1	Distributed Pervasive Systems	78
7.2	Mobile computing	79
7.3	Pervasive Computing	80
7.3.1	Sensors And Actuators	80
7.3.2	Sensor Network	80
7.3.3	Smart appliances	81
7.3.4	Domotics	81
7.3.5	Problems	81
7.3.6	Home Assistant	82
7.4	Smart Vehicles	82
7.5	Why "smart"	82
7.6	Smart environments	83
7.7	Pervasive computing	83
7.8	Current research topics in pervasive computing	84
8	Sensor Data Management	85
8.1	Sensor Devices	86
8.1.1	BLE beacons	86
8.1.2	Sensing device components	87
8.1.3	Virtual sensors	87
8.1.4	Smartphone-based sensing	88
8.1.5	Actuators	88
8.2	Networking for IoT	88
8.2.1	Matter	88
8.2.2	Gateway	89
8.2.3	Platforms	89
8.2.4	Discovery and pairing	89
8.2.5	Device management	90
8.3	Managing and querying sensor data	90
8.3.1	Data acquisition	90
8.3.2	Advance methods	92
8.3.3	Sensor data cleaning	94
8.3.4	Data acquisition	95
9	Context	98
9.1	Understanding context	99
9.2	Defining context	99
9.2.1	Temporal context	100
9.3	Adaptiveness	100
9.3.1	Adaptive video streaming	100
9.3.2	Type of adaptiveness	101
9.4	Obtaining context data	101

9.5	Inferring context data	102
9.5.1	Inference methods	102
9.6	Human activities	103
9.6.1	Deep Learning approach	103
9.6.2	Issues with supervised ML/DL approaches	104
9.6.3	Hybrid data-driven and KB solution	104
9.7	Context representation	105
9.7.1	Flat Model	105
9.7.2	DB-based models	106
9.7.3	Ontological models	107
9.8	Uncertainty	108
9.8.1	Handling Uncertainty	109
9.8.2	Context Management Platform	110

Chapter 1

Introduction

We'll start at 8.45 (on Wednesday) and we have to do some work at home like reading paper and stuff like that.

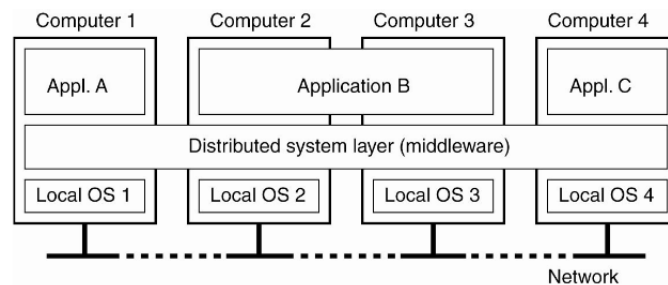
Most of the application as we use are actually running as distributed system as we'll see in this course one goal of distributed system is to hide to users that's a distributed system.

There are not only application, but sometime we'll cover the pervasive system and mobile computing, so mobile wearable and IoT will become soon nodes of distributed system because distributed systems are nodes that will have communication capability (to connect to other nodes) and also computation capability but they couldn't be computer.

The purpose of the course is to give an idea of how the computer system are and will be, that the reason why we'll study pervasive computing.

1.1 Definitions

What is a distributed system? "is a collection of independent computer that appears to its users as a single coherent system". The second part is important, we're using distributed system when we use the browser, even the search engine is a distributed system, but also the DNS is a distributed system (much more hidden for the users) because when we look for something we put the symbolic name on the host that actually doesn't means anything in the network and doing the translation to IP address is done by DNS that is totally transparent and is a distributed system. Once upon a time there was only one DNS server in America but that solution wasn't scalable, the solution was distribute that system on top of a lot of machine all over the world.



But what does means that a system is distributed by software? The idea is to have software that stay above the operating system of machine. We'll always talk something above transport layer of OSI stack (over TCP). The socket are an abstraction above transport layer and we'll use the socket for communication.

Doesn't matter which kind of operating system has the computer (could be linux, windows...) but on top of the OS we have a middleware layer. We use Internet as standard and use the port number for give information to one process on machine, so we'll recognize the host in distributed system by IP and port. There are many ports but they are unique inside a node. When we'll develop the project we'll choose the port that servers will use.

Whatever service is implemented by software that are above the OS of the nodes.

Examples:

- A set of networked PC with shared file system, and process execution dynamically assigned to one of them. The first part is standard using SAMBA, the second part

isn't typically for example we have several computer running word and the system decide that one computer will use the application because it is more free.

- One other example is online games, massively multiplayer online games with co-operating components distributed over different devices/computer. Massively multiplayer online games with cooperating components distributed over different devices/computers.
- The web, it's kind of distributed system because we have many nodes communicating to each other, but we don't consider actually the distributed system for the leak of transparency, because using the domain we can assume where is the machine where the site is. In distributed system we don't want location known, even the name of site is a problem.

Laslie Lamport (invented latex) and give an ironical definition: "You know you have one when the crash of a computer you've never heard of stops you from getting any work done"

If the distributed system is not smart enough to recover from a crash the debugging could be a nightmare.

There is also fault transparency, you shouldn't know when something goes wrong, the system must be smart enough to recover from the crash.

1.2 Goals of a distributed system

If we have to build a distribute system from scratch we have some goals:

- Making resource accessible: resources could be computational power, data, access to something...
- Distribution transparency
- Openness
- Scalability

1.2.1 Transparency

There are different form of transparency in distributed system:

- Location: as we already said all the nodes are in different geographical location but the user can't feel this. hide where a resource is located
- Failure: the system must hide the failure and recovery of a resource
- Access: the computer may be heterogeneous in general, massively in pervasive system. There are computers that have different hardware architecture or represent the number with different number of byte, they basically speak different language. We need to provide some software that are used by different computer, that could be possible using a software that interpret the software for each nodes. For example

exist some middleware software and we can use some API to create a file using some library of google and this API (Google file system) translate the command for the correct file system of node. Hide differences in data representation and how a resource is accessed.

- Migration: hide that a resource may move to another location. Maybe for load balancing, maybe because we understand that are more request from some area and we want to move resource to that area
- Relocation: hide that resource are moving the resource while are used
- Replication: in Google file system everything is replicated even on the same computer, that's important because if something is loose we can recover the information. Transparency means that the user don't know that there are multiple copies of data.
- Concurrency: hide that the resource may be shared by several competitive users. DB (even normal DB) must handle concurrency, similarity in the distributed system happens and the user of DB shouldn't know that happens. DBMS of DB. In distributed system this problem is more difficult to know. The goal is to hide to the user that the resource he's looking for is used by some other users.

1.3 Openness

Something that we want for all of our system, the idea is that we would like to be interoperable it means that if two company build different distributed system they want to integrate them self or at least extend an existing system. We also want portability of the system and extend it.

As developer to achieve openness is to release to open source or making public the interface of our system using some standards.

1.4 Scalability

The distributed system should be scalable. For example a distributed system for online multiplayer game, in the beginning we have few players and after some times we have success and we gain a lot of users and the system must handle more users.

One goal is to choose the right algorithm for scalability.

We cloud have some problem with scalability when we have centralized system, like:

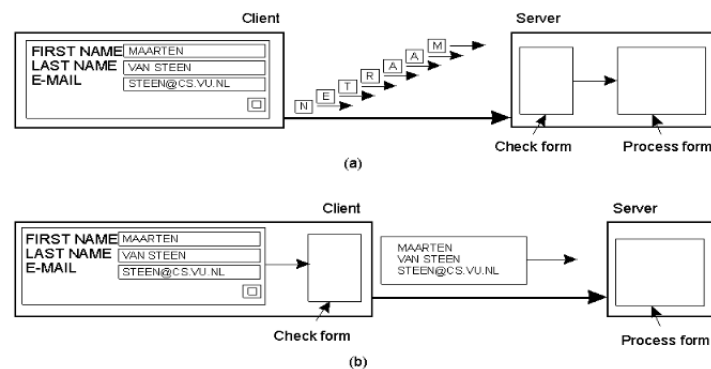
- Centralized services: A single server for all user, what happen if the server is too small?
- Centralized data, is not a good solution for distributed system (like a single DNS file in the beginning of Internet)
- Centralized algorithms, for example doing routing based on complete information. Is a coordinator.

We try to solve this problem and improving scalability with decentralized algorithms that have this characteristics:

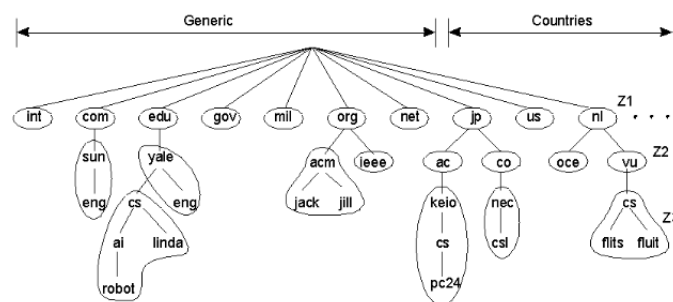
- No machine has complete information about the system state, there is no single node that know the complete status of the network. There is somebody that know about the network in the neighborhood
- Machine make decision based only on local information
- Failure of one machine does not ruin the algorithm. There is no problem if one node goes down
- There is no implicit assumption that a global clock exist. All node has a different clock but there are some algorithms that help us to synchronization, for example exist a server for time that give the right time. We have to keep in mind to not assume that different process have the same clock.

1.5 Scaling technique

We'll see some technique for scaling. When we have web application we can move some server side computation to client side computation and that give us some more scaling because we distribute the work.



Exactly like the people that running Internet in the beginning that divide the domain in hierarchical structure like tree because is impossible to handle all domain in one location, that divide location in zone and each zone has here own servers (DNS).



1.6 False assumptions

We should never assume that:

- The network is reliable, certain things are just impossible in distributed system. We live in approximate solution, also the OS are full of bug and problems and we coexist with that problems. The first used distributed system is Napster.
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator
- Debugging distributed applications is analogous to standard applications

Amazon is actually running in distributed system and use some algorithms that we'll study.

1.7 Types of Distributed System

Distributed Computing Systems:

- Clusters
- Cloud Computing
- Edge Computing

Distributed Information Systems:

- Distributed Databases
- Distributed transaction management

Distributed Pervasive Systems.

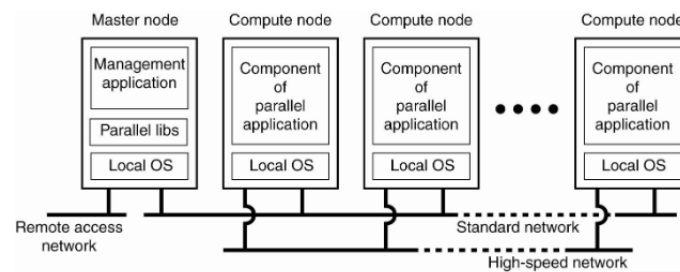
1.7.1 Clusters Computing Systems

The cloud services are running on clusters of computer distributed system but actually multiple cluster connected with high speed connection. A cluster is a collection of similar workstation/servers closely connected by high-speed local-area network and usually running the same operating system.

Before in Computer Science we're building super computer with powerful CPU then the idea of cluster emerging.

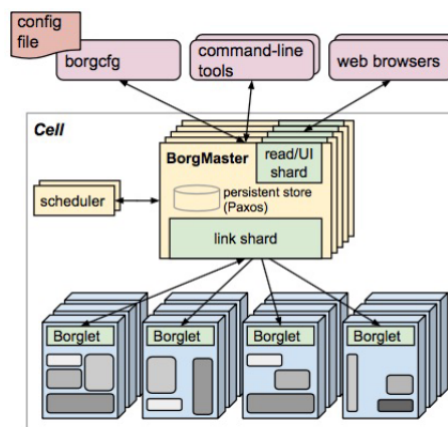
Which are the features of cluster distributed system vs other distributed system? High speed connected, then there are two type of cluster:

- Symmetric approach: every node has the same capability and there is no coordinator. Means that there is no master.
- Asymmetric approach: there is someone that coordinate the work of the other nodes. There is master.



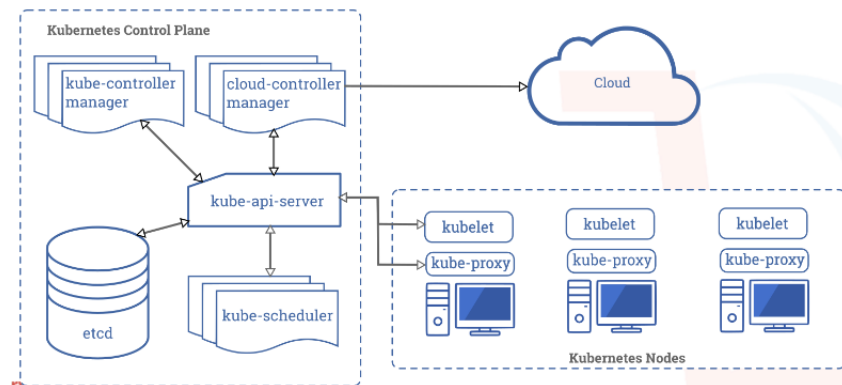
Asymmetric cluster, the node in the left is different from other nodes is called master node that has some specific software that is used to distribute the work to the others.

Example of Bore, they designed this software as an evolution of what already exist.



The problem of Asymmetric solution is that have a single point of failure, if the main crash what happens? also there is a problem of work balance to the master (easily solved by a good design of the system). The solution is to use a distributed system for the master so if one node fail we could use some other machine, but that's not enough because the state of the failing machine must be know by the other nodes so the machine have to share the same data, there is some algorithm to persist the knowledge of the master and all the candidate master (Paxos).

1.7.2 Kubernetes



Is another Google's software to orchestrate the work on cloud and is used by many companies because it is free to be used and is inspired by Borg. It is used to handle containers; Kubernetes is here to move all the Docker applications in different machines.

etcd is a key component where the data for orchestrating is stored and is where, if the master fails, the other one takes the data.

What we have to know is the difference between symmetric and asymmetric approach, not all technologies details.

1.7.3 Cloud Computing

Why is so popular? Cloud computing will not exist if there isn't distributed computing.

- **Resource pooling:** the provider's computing resources are pooled to serve multiple consumers
- **Rapid elasticity:** if we decide to do a start-up and don't know which server to buy with a cloud solution, we can scale very very quickly using virtualization over multiple machines. The idea of computer-like facility as electricity. Be careful of the bill because this cloud solution sends large bills if we're not careful.
- **Measured service:** cloud systems automatically control and optimize resource use by leveraging a metering capability

Model:

- **Software as a Service** like Google Sheets, Microsoft 365
- **Platform as a Service**, like DB, compiler or some other stuff
- **Infrastructure as a Service**, deploy and run arbitrary software (including operating systems and applications) on cloud infrastructure

1.7.4 Edge Computing

All the sort of sensing and adapting technologies in some cases need some low latency so the system should react very quickly to change the environment, a typical example is the autonomous driving, in this case put this data in cloud is not a good idea, we're trying to do computation where the data is collected, like Telecom is doing install computation on the antenna, and like the computation is done on the car.

Edge is realtime data processing at source/on premises data visualization.

Fog is the idea to put the computation where the data is coming from and where the system has to react.

1.8 Pervasive computing

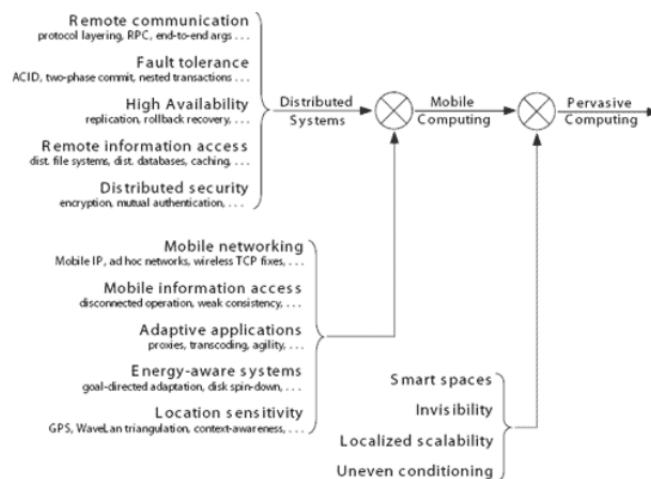
The idea is that computer should disappear, is a bit extreme but the vision is should be use resource everywhere it should be sensing in all the environment and change the environment.

The inventor of this vision says: "the most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it"

the technologies are hiding in the environment and in the future we couldn't be able to see the technologies.

A distributed pervasive system is a distributed system that include unconventional node like iot device, but it should be also adaptivity and prove some intelligence adapting the behaviour to environment for optimizing the system goal.

Pervasive systems are more complex than distributed system, and all together multiplies all the problems.



Doesn't exist a perfect distributed system, because all the transparency are never satisfied, and also does not exist a perfect pervasive system but we can do some example:

- Smart Environment Systems:
 - Smart Home Services
 - Smart energy management

- Smart transportation
- e-Health and well-being systems for:
 - Tele-health monitoring
 - independent living and ageing well
 - accessibility

E-Health is also an important field in pervasive systems, the national health system couldn't handle all the requests and as computer scientists we can help sending people some intelligent system to do some stuff avoid going to the hospital.

Chapter 2

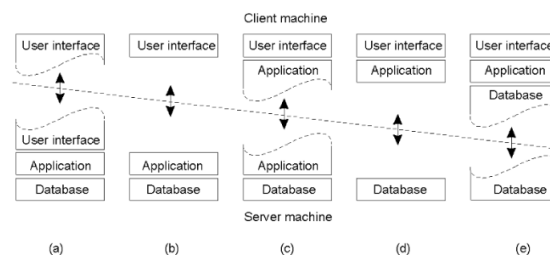
Architecture

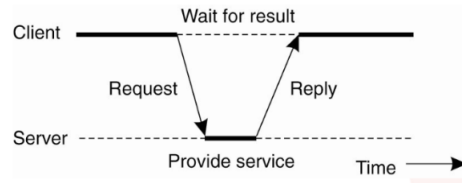
A DS architecture defines:

- The main entities of the system. Every node has process/threads. Some node could be special like sensor nodes. There is a concept of objects and component in distributed system and also there are services, basically functions that can be called.
- The pattern of communications, each node has to communicate with other, the simple pattern is package communication. We have to specify web socket and other stuff related to communication purpose.
- How they communicate
- The role of those entities: some node could be only client, some only server or they could be both.
- How they are mapped to physical infrastructure: How they are mapped to an infrastructure, there is a cluster of masters, replication, caching...

2.1 Centralized Architectures

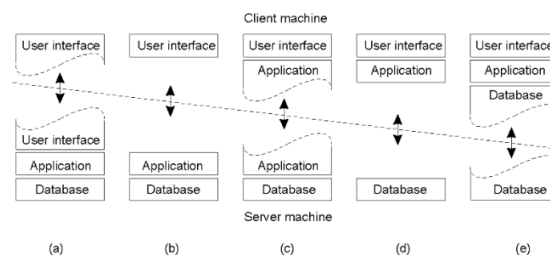
The client-server model is the most common pattern, we have some nodes that run processes that have the role of client that ask other nodes (server nodes) something, they get an answer from the server and continue processing. Also a server node can ask something to another server.



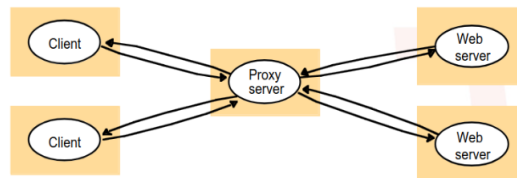


This is a scheme of the general interaction between a client and a server. We have a line upon time, the continuous line means that this is a process running, the dashed line is a process that is doing something else or even not running, the arrows indicate communication. The arrow's line are inclined because of the network latency.

We have to keep in mind that even if our choice for architecture is client server there are a bounce of possibility for this architecture. In the picture we have two extreme end some in the middle a lot of intermediate solution for client server architecture.



Very often there is something in between, maybe there is a proxy server, the role of the proxy is caching. If many clients are asking to the server the same thing very close in time and the proxy is closing (geographically) to those clients, we avoid going to the web server and get the answer directly from the proxy. Proxy can have also security role because they can check the contents of the messages.

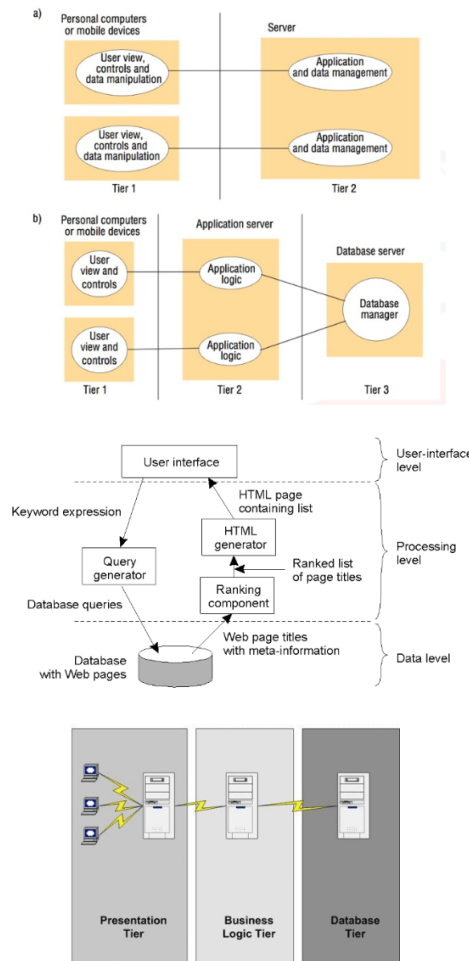


We may have multi-tier architectures, very often there is a 3 tier architecture, DB database server, application server and personal computers or mobile device. DB server could be replicated. Also the reason why we put DB on different machine is because that machine couldn't accept communication from Internet, only from the certified machines.

An example of application logic is search engine, the application logic is inside processing level and of course this software (query generator) take a natural language input (a web search), make a query for the DBMS, that return the web pages to a component (in application logic) that make the rank of the page, and then there is a HTML generator for generate the actual list of web pages.

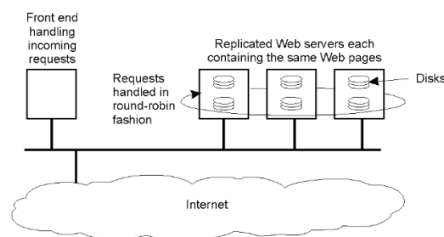
2.1.1 Vertical distribution

Assign different nodes different functionality. In the example for the database tier we have a machine, for the business logic we have on other server and the one that is doing HTML, graphics goes to an other one server.



2.1.2 Horizontal distribution

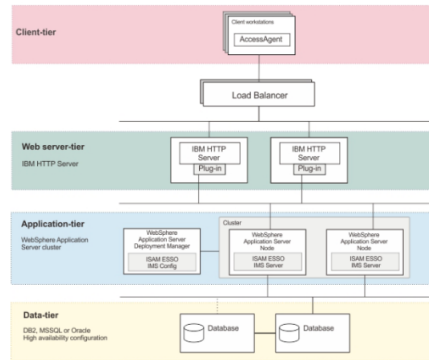
The same functionality is distributed on multiple servers/nodes with load balancing. Like we said about Borg.



The example that we see is about the web server, we may have a lot of request from web site but not allow the request must be solved on the same machine, we have a load balance server that distribute the work.

2.1.3 Vertical and horizontal combined

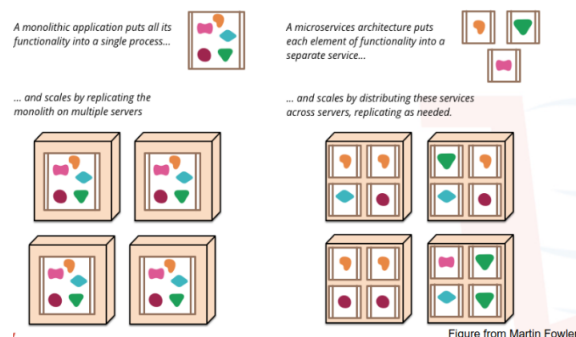
We can design s system that is both vertical and horizontal. Each functionality duplicated on a separate group of servers with load balancing.



2.1.4 Microservice architecture

The modern way to design distributed system is with microservice, and it's an extreme vertical distribution (docker). We have one component that do one specific thing, we want to running each component/functionality of the application logic in a separate process, possibly on a separate machine.

Micro service must be independently replaceable and up-gradable packaged with all they need to be deployed. It's a service because someone else should call this functionality when he need it. The overall idea of DS is to have several copies of each service for fault tolerance, scalability. micro service communicate with lightweight mechanisms (REST API) focused scalability: act on the specific services that need to be scaled, not everything must be scalable. micro service may be written in different programming languages and by different teams.



On the left we have a monolithic application, when puts all its functionality into a single process and scales by replicating the monolith on multiple servers.

On the other side there is the idea of microservice architecture, we separate each functionality within each application and run it separately. If one service need some other stuff from other service they communicate on the network, even if they are on the same machine. And scaled by distributing there services across servers, replicating as needed. We can decide how to distribute the services in the systems.

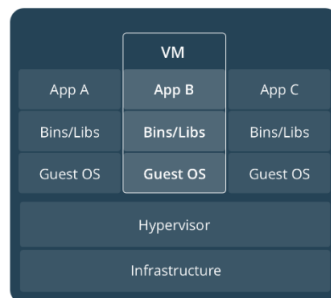
2.1.5 Container

It's an abstraction at the application layer that packages code and dependencies together (package everything that is needed from a single function). Virtualization at the process level in user space.

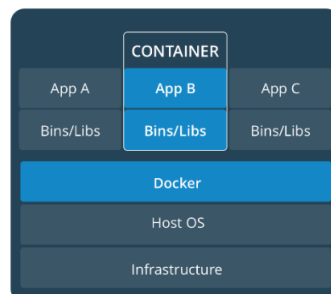
The goals are:

- offering efficient support for the micro-service architecture
- increasing the portability, enabling easy migration
- reducing problems at deployment time

Containers are the virtualization of the process level. Having a vm is like having a machine that doesn't exist. On the same nodes I have some software that create vm that is hypervisor and on top i create my virtual machine.



The idea of container is having nodes with operating systems we install the container platform (container engine, Docker is an example of a container platform) and on top of it we can run many services on the same machine. We virtualize only the process level, not all the machine.



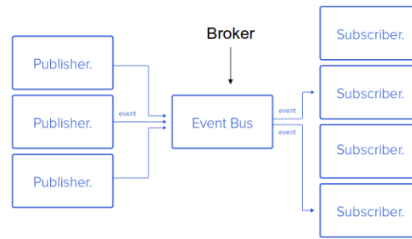
Different instance of an app that runs on container share the same environment.

2.2 Event Bus Architecture

The architecture is not based on the client that ask for a service, there is a kind of intermediate node (could be virtual) that process the communications about that something happened to some other node that are interested in that event.

For example using sensor, we have some sensor in an ambient that see if something happens, this are events and the nodes, called publisher, publish this event to the broker (bus). But each events is different (security, temperature, movement...) and each subscriber is interested in one type of event.

This is based on publish-subscribe communication pattern.

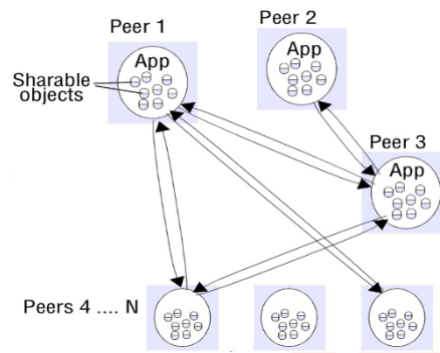


2.3 Decentralized architectures

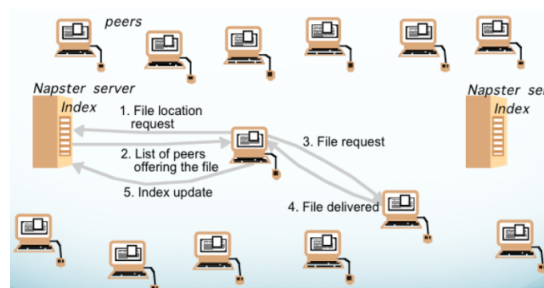
We write a single piece of code and make it run in different process.

Each node in peer to peer system have a part that is client and other that is client.

They are designed so that each user contributes resource. Operations does not depend on any centrally administered system (no coordinator).



Napster was the first peer to peer system, they want to exchange music, and it's working so well and big company put it down quickly. It's not a pure peer to peer system because there are 2 nodes that are different from the other, server index that keeps track in which nodes there are some nodes, one nodes that want a song contacts this server this server gives to that node a list of peers (ip numbers) offering the song than this node actually takes one of this addresses and contact one peer that give the song and than the nodes communicate back the updated file to the server (because now it has the song too).



The key problem for the system is how to place data objects across many hosts in order to:

- Achieve load balancing while accessing data, proportional load to the capability of each machine
- Ensuring availability avoiding overheads, every resource must be reachable anytime and anyway

There was three generations of P2P systems:

- First file sharing systems: napster
- Protocol are refined to improve scalability, fault tolerance, anonymity: Gnutella has the goal to be scalability, FreeNet wanted to improve anonymity, bitTorrent wanted to improve scalability the idea why should exchange the whole thing? we divide the different resource in chunks and take the chunks from different peer.
- P2P Middleware

2.3.1 Middleware

There are three goals of this system:

- Enable clients to transparently (we don't want to know the other nodes, we just want the files)
 - locate and communicate with any resource, we need to find other nodes quickly
 - add and remove resources, if we insert and remove files we have to insert and remove quickly
 - add and remove peers

If we want to compare different system we'll do basing on this things, how quick the nodes reach each other and communicate? how quickly a resource (and node) is added and removed to the system?

Overlay network

The idea is to build a virtual connection between peers, the overlay network. Optimization criteria: global scalability, load balancing, locality of interactions, support of encryption and authentication, anonymity. P2P systems are often organized in an overlay network: a network in which nodes are formed by processes and links represent possible communication channels. An overlay network is a logical network over an existing lower level network.

There is a routing overlay, it's a distributed algorithm which goal is to provide quick resources in the overlay network. We'll organize our node in a ring, each node know the address of the successor in the ring. The communication happens by message passing along the network.

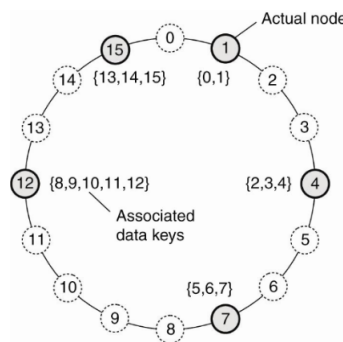
Routing request from clients to hosts holding objects of interest at the application level instead of the network level (IP). Is based on global identifiers and should direct to one of the replicas of the object. We'll handle the problem of names of the nodes, because we assume the idea to have global identifiers. Routing overlays protocol also deal with insertion/removal of object and nodes.

There are two different categories of overlay networks:

- Structured: overlay network deterministically build (often via hash table) in order to obtain efficient routing towards the node containing the required data. It's focused on find resources quickly
- Use randomize algorithms. Each peer knows only its neighbors and we only talk with the neighbors. Sometimes they have hierarchical structure through superpeer nodes. There is not certain of finding other nodes, there is a probability.

Main goals of structured overlay is quickly mapping a resource to a node, where to find a file (address of the node that stores the file).

The main idea is to have distributed hash table (DHT).



In the example the overlay is a ring, the shaded node there are nodes available but there aren't process running on these machines, we need a global identifier and we use a identifier based on bytes, in this case there are 16 nodes (this addresses space should be big as much file we want to handle). In binary this is a 2^4 addresses, node will share this addresses with resources, this means that if one new node would participate to this system we give it an address using hashing functions, hashing IP and port, and how can i get i file name, hashing the file name.

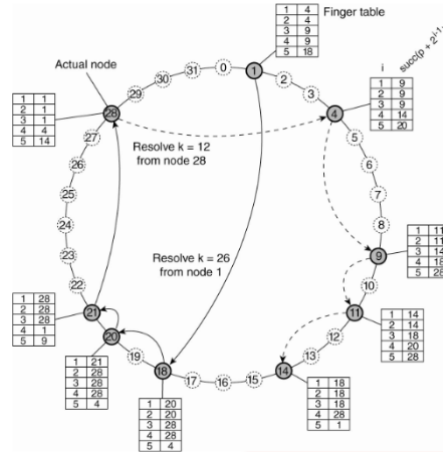
We give numbers in this address name both to nodes and files.

- fixed address space (up to 160 bits)
- Each node gets as id an address in the space (for ex by hashing its IP&port)
- Each data item gets a key (address) in the same space

A data item with key k is managed by the first node with $is \geq k$, called $\text{succ}(k)$.

In this ring topology how can write a search for the files? the only way we can do this resource is to ask to ask to one that i know, that is my successor and so on. The complexity is linear $O(n)$, if we have million of nodes is not good. This is way distributed system have hash table. We want logarithmic to the number of node.

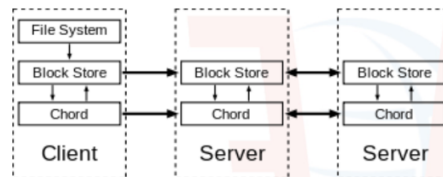
The idea is to give some more information to each node, each node will have a small table, called finger table (key resolution, means to find the address of the nodes that actually handle the files). Each table have m number of entries, m is the number of bits of the space (in this case $2^4 - > m = 4$), the idea is to have shortcut to other nodes in the ring. What's inside the table? the element in the table stores the id and address (IP, port) of $\text{succ}(p + \{i - 1\})$ where p is the current node and i is the entrie of the table. Remember that succ means the first node with k grater or equal then. In the first entries we always find the next node.



Resolving key 26 from node 1 means that we want the file that has hash value = 26 and we start to find it begin from node 1, the node 1 doesn't have it but thought the table he can go suddenly to node 18.

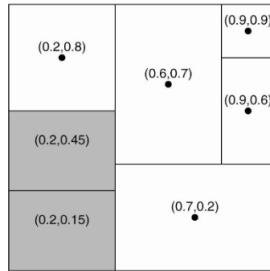
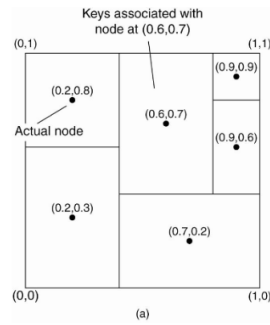
Like middleware we have also to provide node insertion and deletion. A join or leave requires several messages in the network $O(\log^2 N)$. Of course we have to maintain the hash tables each time the network change, we have a cost to maintain the network. One of the reason why the people use random overlay is because of this cost.

We like system like this because we can build a distributed file system.



Just to give an intuition of on other architecture is multidimensional structure called CAN, in the middle of region there are coordinates, these coordinate are used to find files that gone in this region of the bi dimensional space. Is the notion of closeness in multi dimensional space.

When we have new node we have to fragment the bi dimensional space, inserting and deleting nodes may be difficult.



Limitations:

- Maintenance of complex overlay structures can be difficult and costly to achieve (specially in highly dynamic environments)
- Need for self organizing nodes, naturally resilient to failures. If one of those node fails people trying to contact this nodes, there will be time outs in this communication that communicate that something happens on that node.

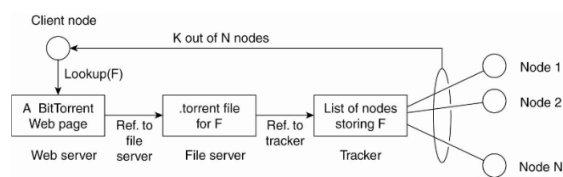
Unstructured overlays

Routing is based on randomize algorithms:

- Each node stores a list of neighbour nodes randomly built
- Resource are also often randomly assigned to nodes
- Search for a resource starts from a node and propagates according to local views
- Search is often limited to a number of hops or timeout. Resource replication helps improving search success rate.

2.3.2 Hybrid architectures in Collaborative Distributed Systems

A real system probably can have also a centralize parts. It's hybrid because bootstrapping using a client-server protocol, example of bitTorrent. For instance a client ask for a file, a torrent file that have information, especially we have references to trackers, that are a list of nodes storing chunks of file. Each node that have chunks act like client and server because accept request from other nodes.

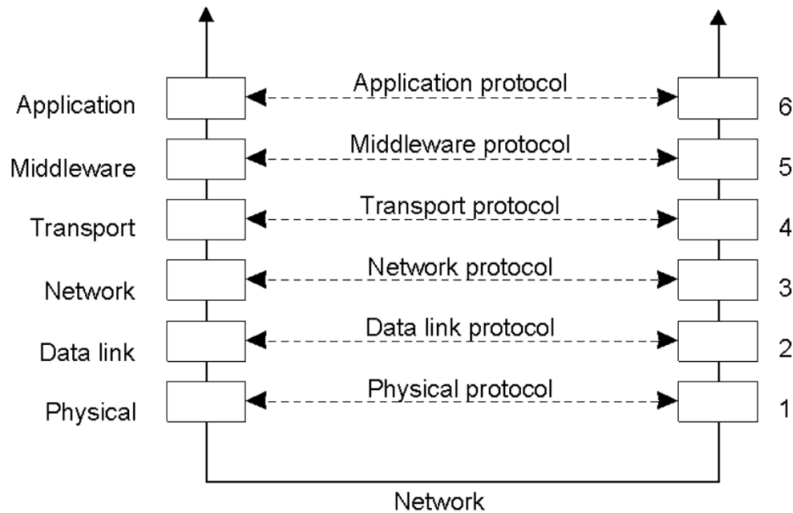


Chapter 3

Communication

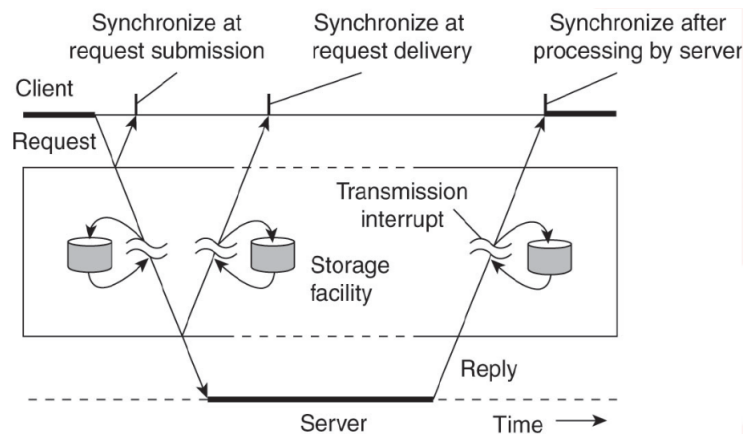
Of course we need a efficient communication in a distributed system. We'll cover the basis of communication protocol in order to be able in the future to understand the new one.

We are considering always the middleware and application layer in ISO/OSI stack.



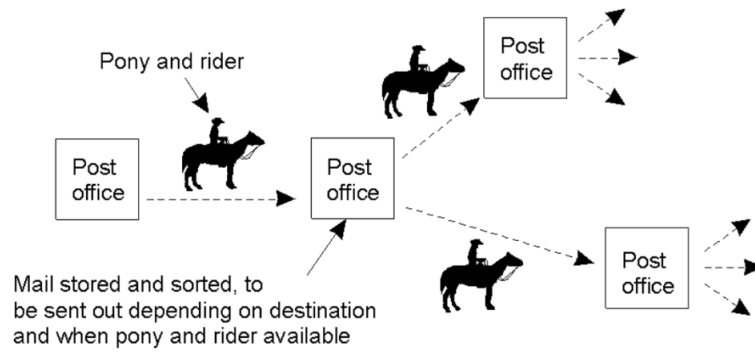
3.1 Types of communication

Typically TCP/IP is the transport protocol used by what we're explaining later. This figure show us that are some different point of synchronization, the middleware of distributed system may offer some functionality.



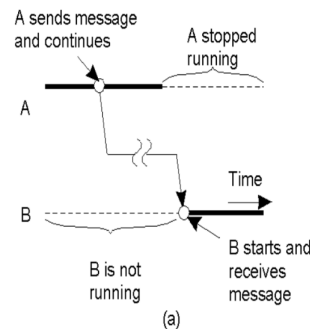
The simplest way to understand is thinking about a messaging system, if we are not in the other side of the messages send there is an infrastructure in the middle that take the message for us and give it to us when we are ready. This infrastructure give us persistency and synchronization. In this case the request is stored in the middleware, and the middleware give some communication to client to keep synchronization.

We can make the same example having some physical post office that take the post and send to other people.



We have to distinguish very clearly feature of synchronous and asynchronous communication. A transient communication is one that not offer persistent, for example when a server is down we lose the message.

Here we can see the different flavour of this things.



A is the combination between persistent and asynchronous, the drawing in the middle means that the message is stored. This is asynchronous because A keep running also if it doesn't receive a response from B.

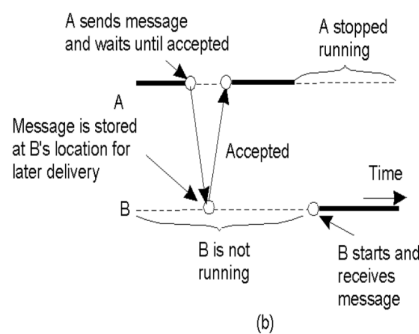


Figure B is persistent but synchronous. A sends message while B is not running, in this case in B there is kind a different process that has persistent storage so it keep the message from A. In this case process A is happy to know that his message got to the node B. A will wait until receive the information that B received his message.

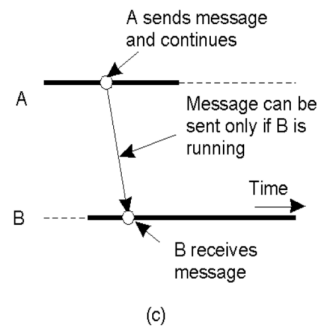


Figure C is transient, there is no way to store the message not by a middleware neither by the receiver. This is the typical interaction when we make a request on the web. It's also asynchronous, that means that process A continues to do whatever to be done and not wait the answer from B.

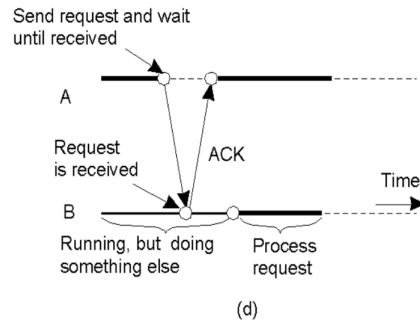


Figure D is transient, however we have synchronous communication. This is the most common actually. The difference with figure C is that process A is really waiting for the answer. In this case process B is running but is doing something else, so give an ACK to A to say that he received the message.

The next figures will be transient and synchronous, they'll have different ways to do synchronization.

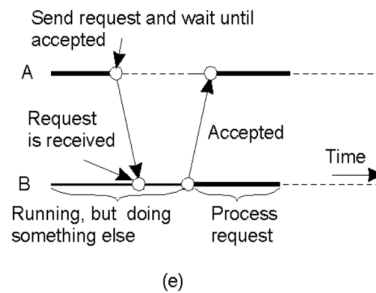
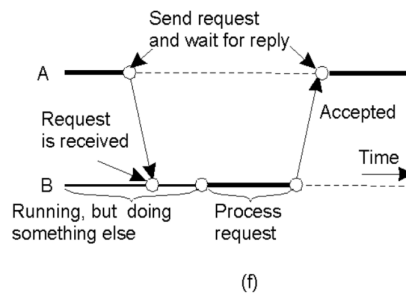


Figure E, process B sends an ACK to A when B is paying attention to the message that A sent to B.

Figure F, this is the case when B actually keeps waiting until B gives the answer.

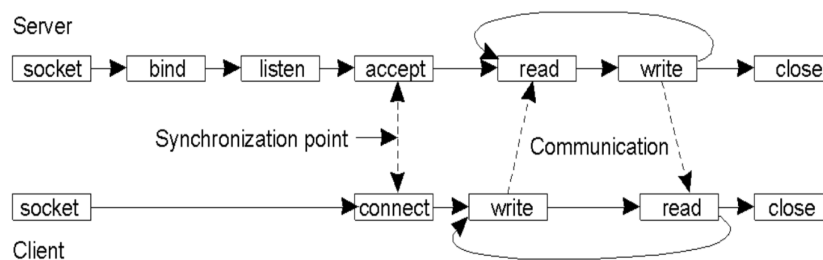


3.2 Message Oriented Transient

It's a kind of communication middleware. There are some primitives to use sockets.

Primitive	Meaning
Socket	Create a new communication endpoint
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

We may think about communication as a cable, there are two processes in the extremes of the cable, we have to plug socket for communicate with other process. In communication between two process we need two sockets.



Sockets are treated like files, they have socket description (like file descriptor in which we have the id of the socket for example we can have the id for standard input, output and error), so OS read and write to these file.

The server do BIND, actually associate the port to the socket. This happens because the TCP/IP protocol doesn't know anything about the file descriptor that we give to socket.

We also do LISTEN, that say that we are ready to receive message, this allocate some resources in the OS, usually create a queue. There is limited space.

Then we are call ACCEPT, is actually the point in which there is the connection and read and write go. Block caller until a connection request arrives.

In the other side there is a SOCKET, we don't need the LISTEN but BIND is done automatically, the reason why we don't call it is because the number of port isn't really important, the OS take the first that is empty.

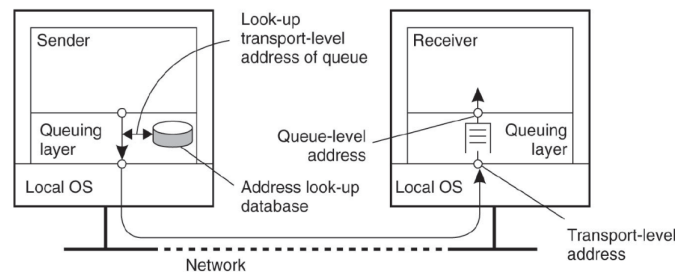
Client call CONNECT that attempt actively to establish a connection.

After CONNECT the server is writing a reading using the file descriptor (so treat socket like a file). The same happens on the client.

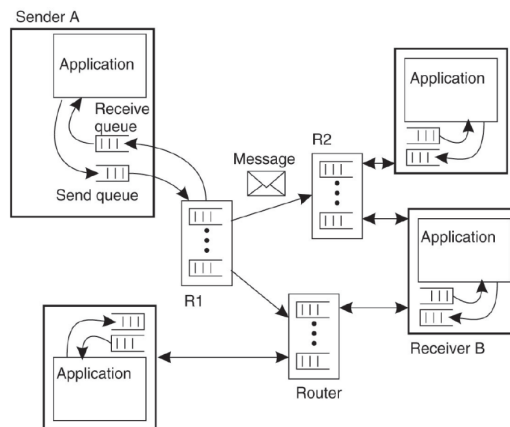
This is transient, there is no way that the connection is stored in some way.

3.2.1 Queuing Systems

If we want persistence we can use queue. Queue systems are more sophisticated, compared to what we can see in the example, because some other node have the ability to store messages in queue.



In the example we have 2 processes, sender and receiver, there is a middleware that actually handles some storage, the queues. This queuing layer is not necessarily in the process of client and server. We write and read on the queue, so the queue must have an address.

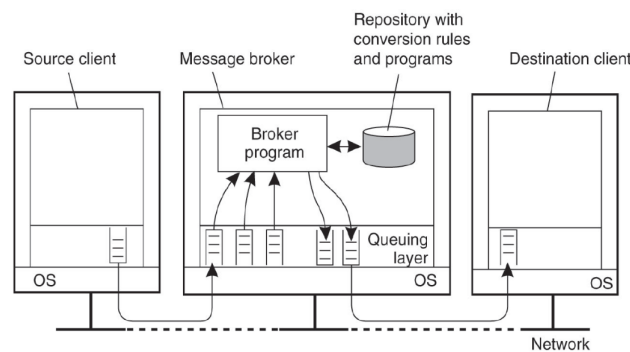


In the example with different nodes there are nodes called routers. Inside them there is local queue and a queue on the routers. Very similarly to what we do with the mail server. This routers in distribute system may have different roles.

Primitive	Meaning
Put	Append a message to a specified queue
Get	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages, and remove the first. Never block
Notify	Install a handler to be called when a message is put into the specified queue

Message Brokers

The routers can also offer transparency, because in these nodes could be there a repository with conversion rules. So these node are the ability to receive messages from client that speaks different languages.



When queuing systems?

- When asynchronous communication is preferable
- When scalability can be increased by admitting delays in requests and responses. Communication in real time this persistence doesn't do the difference.
- When the producer is faster than the consumer
- When implementing a publish-subscribe communication pattern

Messaging protocols

- XMPP: used for instant messaging
- AMQP: ISO standard, general purpose. It's suitable for a broad range of messaging-middleware infrastructures, and also for peer-to-peer data transfer. It supports message queues.
- MQTT: is designed for IoT framework because is lightweight. It doesn't really support queue because there aren't intermediate node (only one, broker). The messages are sent to broker and then delivered to subscribers so it use publish/subscribe model.

They will never ask details but there is a lot of material online, and we'll go deeper with MQTT in the laboratory.

Some popular message-broker software:

- Eclipse Mosquitto
- RabbitMQ
- Apache ActiveMQ and Kafka
- Google Cloud Pub/Sub
- Amazon MQ

RabbitMQ is an implementation of AMQP, that's why is used in industry. It consists of:

- A broker, that is in the cloud
- Clients: written in different languages
- Clients communicate with the broker through AMQP

We can put a tag in the message and the message will be put in different queue.

CoAP

If we need a lightweight protocol (for IoT systems) the choice are MQTT and this other one, CoAP.

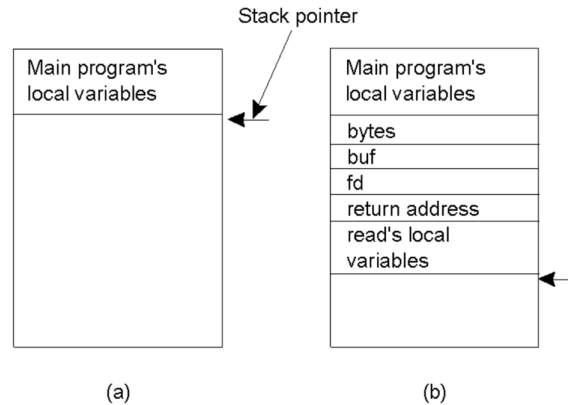
This is not designed along producer consumer model, is more related to REST. Would like to keep the client server model (request and answer) but it's very efficient. We use URI to identify resources and REST model. Is web based. Instead of TCP here is used UDP because is more efficient.

3.3 Remote Procedure Call

It would be nice to have an abstraction for programmer that give us some functions and these functions can be called remotely. It is something that hide the complexity under the hood.

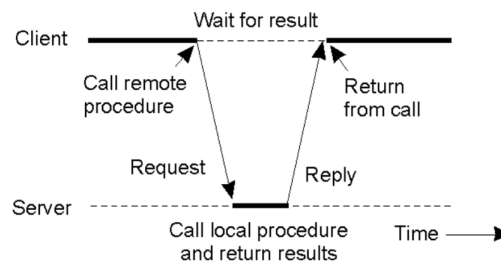
3.3.1 Conventional Procedure Call

We start considering the conventional procedure call in our PC. We have the stack and we consider the simple function read that has in input: fd (file descriptor), buf (to put inside what i read) and nbytes (the number of byte that i read). We put all the stuff in the stack.



3.3.2 Client and Server Stubs

We have a client, we call remote procedure and something in the system has to do the request (stubs). In the server we call local procedure and return results, then the server reply and give client the return from call. (transient and synchronous).

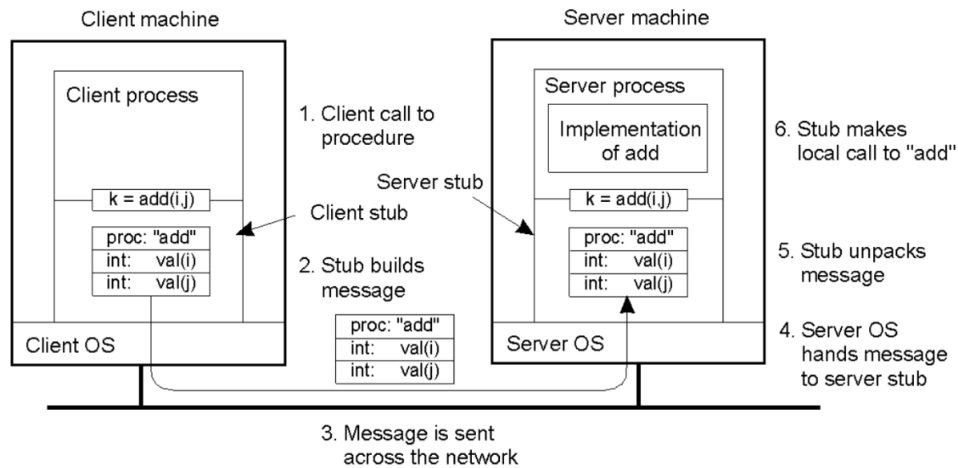


This is what we would like to achieve but we want that the communication will be transparent.

Stubs is the piece of software that implement remote procedure call. This is a piece of software that the middleware will write for us.

Steps:

- Client procedure calls client stub in normal way
- Client stub builds message, calls local OS (because TCP/IP is in OS)
- Client's OS sends message to remote OS (using TCP/IP)
- Remote OS gives message to server stub
- Server stub unpacks parameters, calls server
- Server does work, returns result to the stub
- Server stub packs it in message, calls local OS
- Server's OS sends message to client's OS
- Client's OS gives message to client stub



- Stub unpacks result, returns to client

The client build the message, in this example put in the message the name of procedure and the value of variables that want to pass. Then put the message on the network. Whoever want to offer a function in distributed system has to publish the interface to expose the type of parameters that accept. ServerOS receive the message and pass it to server stub (application layer over TCP). The stub unpacks message and makes local call. The result does the inverse travel. Unfortunately it's not always this simple, sometimes there are data structures and not simple variables. Also we can use reference of variable in function, we need to modify the data structure. The main problem is that we cannot use any reference.

3.3.3 Marshalling and Unmarshaling

Parameter marshalling is the procedure to format RPC parameters in a message. Parameter unmarshalling is the inverse procedure of extracting the parameters from the message. Marshalling requires the serialization of objects, i.e. their coding in a sequence of bytes. Serialization can be in binary (like in gRPC and Java RMI) or in strings (like REST WS in JSON).

Someone have to implement the conversion of languages in order to guarantee the access transparency.

We have the example of passing value from reference, client stubs goes to memory take all the data of the array, put in the message and send all this serialized stuff to server.

```

foofoo( char x; float y; int z[5] )
{
    ....
}

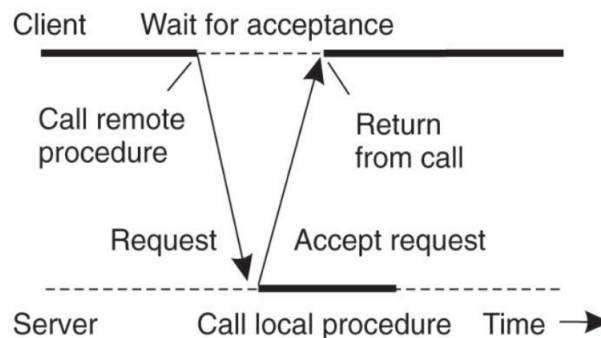
```

(a)

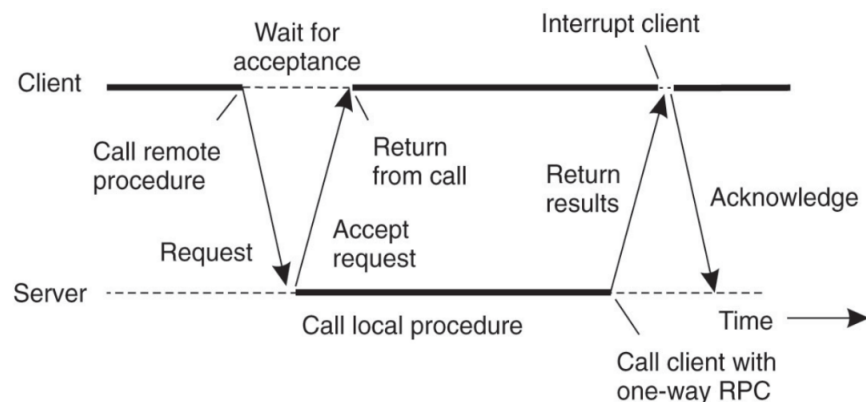
foofoo's local variables	
	x
y	
5	
z[0]	
z[1]	
z[2]	
z[3]	
z[4]	

(b)

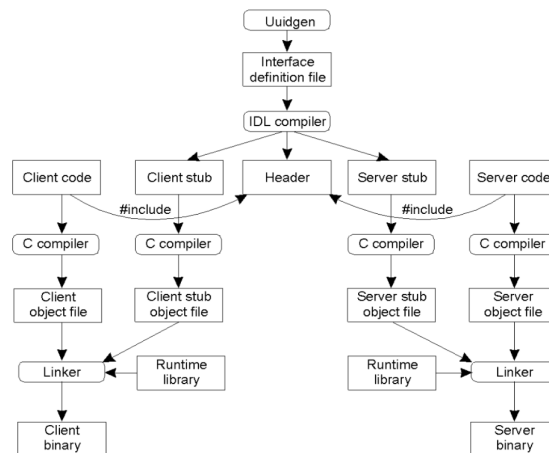
RPC can be also asynchronous (may one question). In this example client is waiting for the acceptance not for the result.



When the answer is ready we can send to a client an interrupt (fake interrupt because we are not in the same OS).

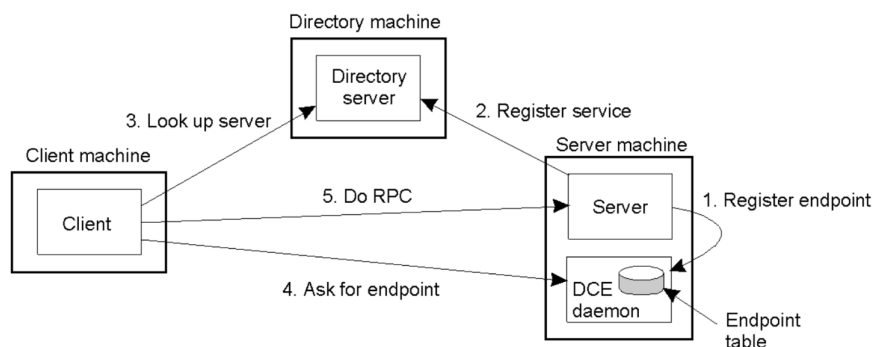


3.3.4 Writing client and server



Usually programmer is have to is define the interface of functions, in general this interface is written in neutral language because in distributed system every node is not obligate to use the same programming language, it's cal IDL. IDL offer us a list of compiler for different language and than in every system have our code and stub code. If everything work correctly programmer hasn't to do something. gRPC is an example (that we'll use) of client-server RPC structure.

3.3.5 Binding a Client to Server



We could have different server that give different services. So client could ask to a directory server which server offer the service that we want. The server has to register the service on directory server, it give only the IP address because in the server there is a daemon that is always up. But it's possible that the server is not running, so in one process that is always running (daemon DCE), this daemon check if the process is not up and running it will wake up.

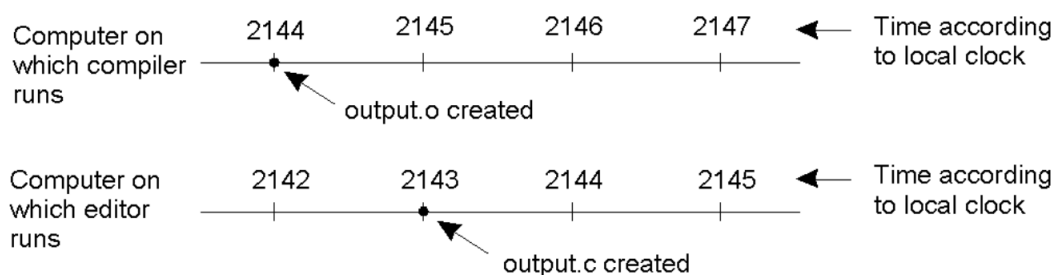
Chapter 4

Synchronization problems in distributed system

It's important to talk about synchronization because several nodes in distributed system want to cooperate to do a job. We already seen the way of doing synchronization using coordinator, we need to understand how to do it without a element that receive message and tell to the other what to do.

Clock synchronization is the simplest way to do synchronization, each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

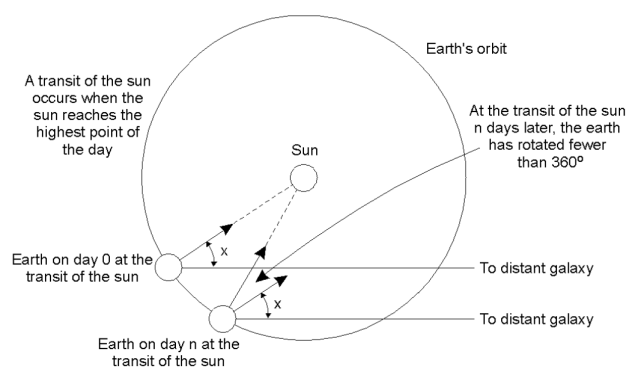
If we have several files in our programming project, when we have to compile the program in C we use the call make that recompile only the files that have been modified. This tool ask the time when the files were compiled and when they were modified in order to recompile only the once that have been modified. The problem in distribute system is that two editor in different nodes could have different time.



In the example we have in one line a node on which compiler runs and in the other a node on which editor runs and they are not completely aligned. At one instant the editor use the command make that compare the time when the project was last compile and find that the time is previous of the time of the call, so it don't recompile because we have a mismatch of the time of two computers.

So try to make all the clocks of the nodes synchronized. We need to understand what time is it. In computer science we use UTC, and in distributed system we want that all the nodes knows UTC.

Before to talk about synchronization using clock we need to understand the basis of UTC. Time is actually synchronized with astronomic events, the basic unity of time is the mean solar day (I see the sun one day and see it in the same position the day after). Astronomers define the mean solar day and then they defined time units based on this observation.

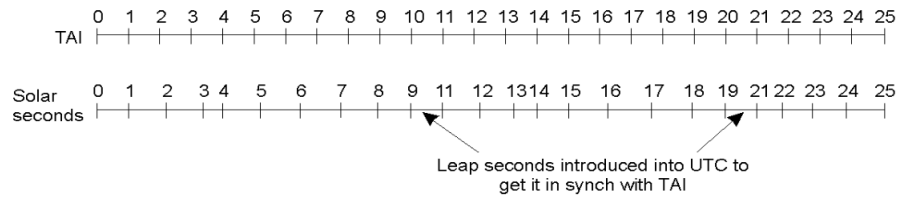


Something has to generate an event every second, that's the frequency that must be aligned with astronomical observation. UTC is not atomic time, because once in a

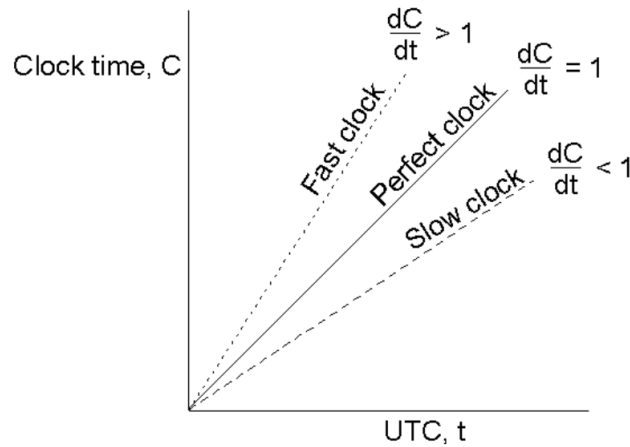
while time institution tell us that if we want to keep align our time to what happens in the universe we have to add (or remove) seconds (we cannot predict when the time will change). So basically UTC is atomic times corrected using leap second tells by time institution.

Atomic clock is a device that uses an internal resonance frequency of atoms to measure the passage of time, it usually use cesium.

TAI (International Atomic Time) is told broadcast from time organization (based on several atomic clock), and they introduce leap to keep our time in phase with the sun.



The reality is that we have the desire UTC time and the time observed by the node, in general we have a clock that go bit faster or slower than perfect UTC time. In this case we can observe that more time passes more will be the difference. Depends on this number we can decide how often to synchronize UTC time. In distributed system we have to decide when to synchronize to UTC, it's based on how much precision is required from our system.



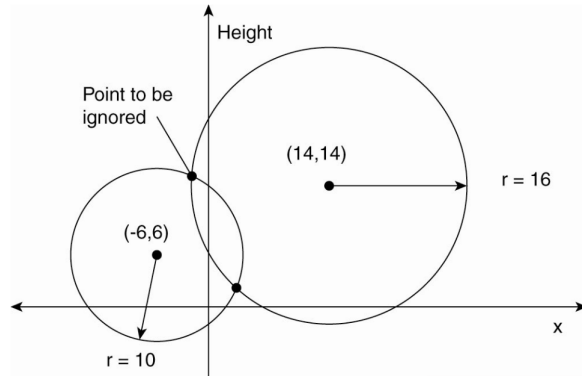
Perfect synchronization is impossible, we can have good synchronization. This is related to network latency.

4.1 Synchronize physical clocks

4.1.1 GNSS

One possibility to get UTC is to use GNSS (GPS). There is a network of satellites and they have atomic clock on board so they know the time very precisely and they continuous broadcast signal, in this signal there is the time. When the receiver receive (not ask for the time) the messages from satellites we can see the time, the position of satellite is known.

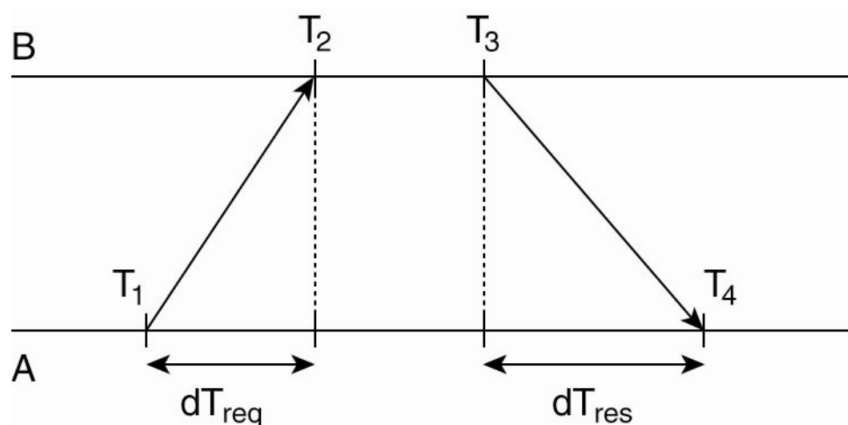
What I know is that we have difference in time of our time and the time of satellite we can calculate the difference and know a circumference, if we receive signals from different satellites we can know our position using the intersection of the different circumference. This is for position, time is only another dimension of this problem.



4.1.2 Network Time Protocol

There is a network of time servers that implement a distributed system. Because there is not only one time server, there is hierarchical system of time server, the hierarchy is based on the fact if the server has an atomic clock or not.

Christian's algorithm, A wants to synchronize the clock and make a request to B. The tricky problem is how to evaluate the answer of B, because there is a lot of delay in network connection. The trick is to repeat this request several times and try to evaluate the delay. In the same message when A sends the request, it also puts the value of his clock. When B receives the message, it puts on the message the time when he received it (T_2) and the time when he sent the message (T_3). In this way B can know the delay and has an idea of the precision.



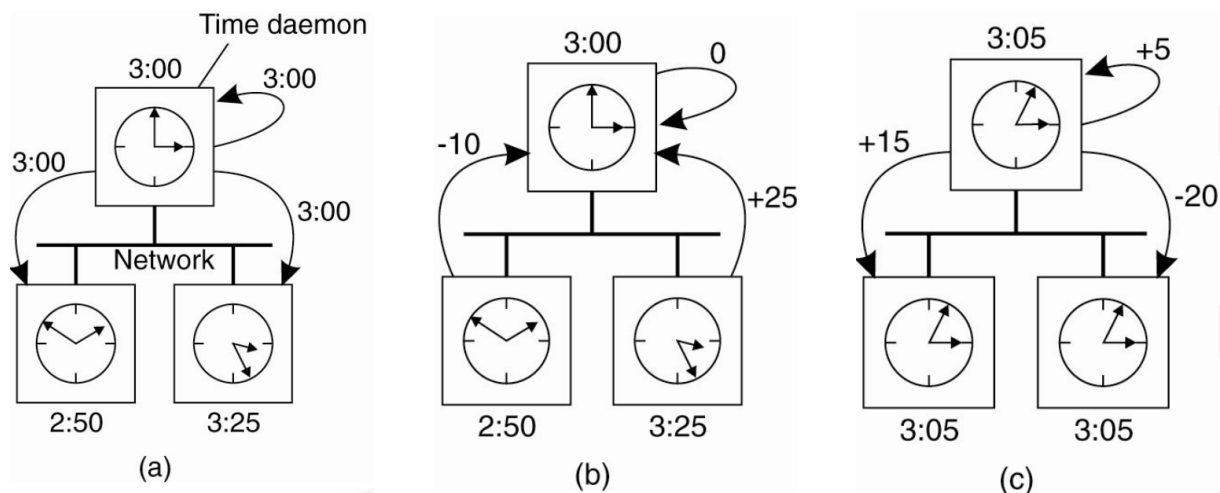
But how often should I call this procedure? usually in our laptop we don't need a lot of precision so call it every 24 hours.

4.1.3 The Berkeley Algorithm

We can use this method when we don't care if our system have the same time with all the world. (remember that this is not based on UTC, typically question at the exam)

There is a coordinator, here is called time daemon, that say to other nodes to synchronize with his clock. Daemon looks to its own clock and send a message to other saying his time, the other nodes answer saying how much they are off to the time that they receive.

Than daemon tells to each one how much they should adjust the clock (time is an average to the time from the nodes), it takes the average from the clocks, going backward in time is a real problem for the file system we can imagine (irl slowing down clock).



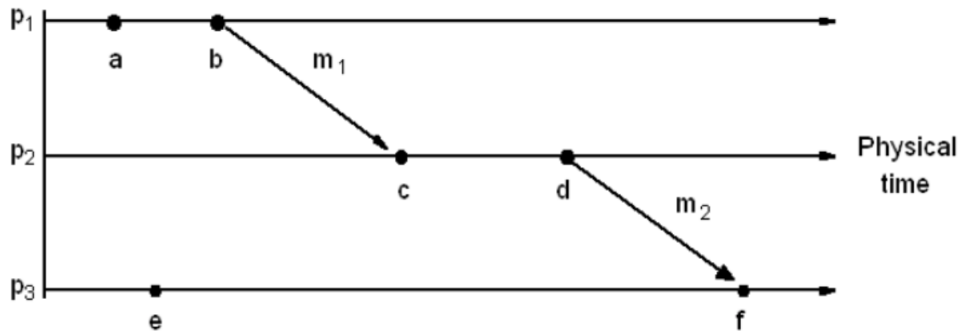
4.2 Lamport's Logical Clocks

In many cases it is not necessary that nodes agree on their physical clocks value. We would like to share among the nodes the knowledge of partial order of events, in order to understand that the last modification of the file is before or after the last compiling. Basically we don't care about the time but only about the order of operations.

An event can be an internal event (something that happens in a single process), or a message received or send, we'll use the message as event.

The idea is to have a counter in the system used at each node as a logical clock. It's incremented every time an interesting event occur (usually sending or receiving messages). So considering a given instant in real time, the logical clocks on the different nodes may have different values.

If event a is before event b , than the logical clock assigned by the process when a occurs is smaller than the logical clock when b occurred. (es. receiving a message should be after the event of sending that message). Goal: if $a \rightarrow b$ holds, then $C(a) < C(b)$ should hold.



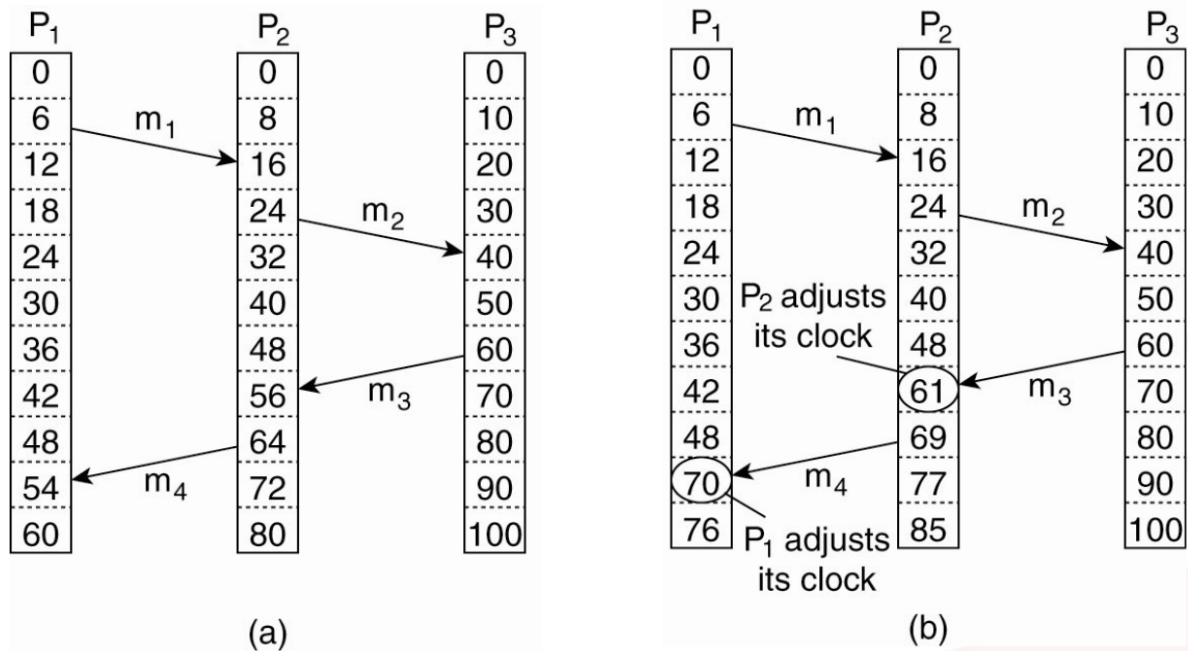
$a \rightarrow b, b \rightarrow c, c \rightarrow f, e \rightarrow f, \dots$ We cannot say $e \rightarrow b$ or $b \rightarrow e$

Let's see an example. a, b, d, e, f are events, the arrow is the flow of physical time. We can say that a is before b because they are on the same process like c and d . We can also say that b was before c (like d and f) because they send and receive the same message. We cannot say anything about e, b and b, e .

If a is the event of sending a message m by a node and b is the event of receiving m , then a occurs before b , we want to ensure that the logical clock of the process a is smaller than the logical clock of b .

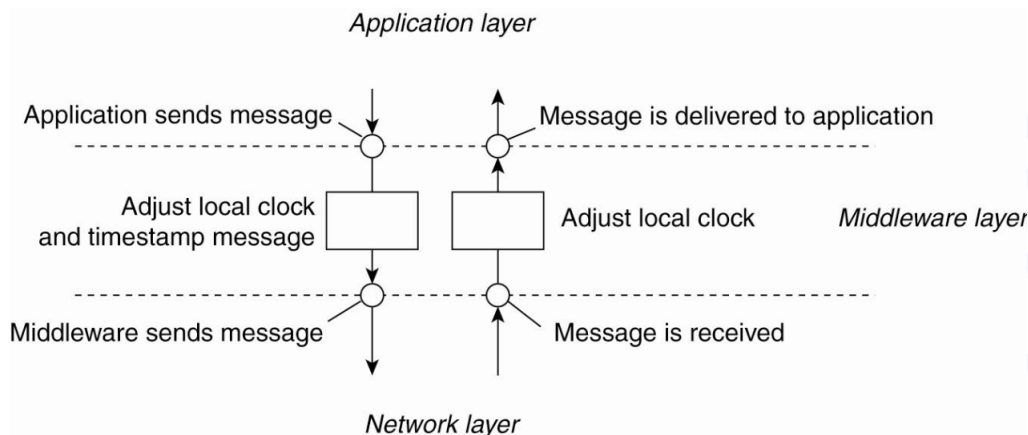
We have three processes, each with its own clock. The clocks run at different rates. When we see that the logical clock of the receiver is greater than the logical clock of the node that sent the message, everything is OK. The problem is when the logical clock is less than one of the sender, in this case the receiver has to adjust his clock. The minimum adjustment that he can do is adding one (56 becomes 61), after this modification all the next events will be increased by the difference of the time after and before (64 becomes 69).

To do the increment in the message I write the value of my logical clock.



Modification of logical clock must be hidden from developer perspective and must be implemented in middleware. The message comes to middleware that adjusts the local

clock.

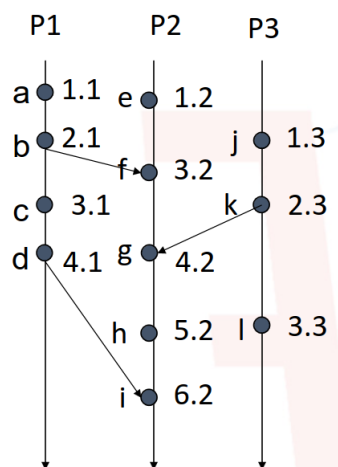


The algorithm we've seen deals with modifying the partial order of clocks. If I don't want there to be equal logical clock values, one of the tricks available would be to assign names to the participants of the distributed system. We should append to the message the value of the logical clock plus the unique name of the process, es $T1 = (C_i(a), i)$ where i is the process number. At this point, when comparing the logical clock values, if they are equal, we compare the process numbers.

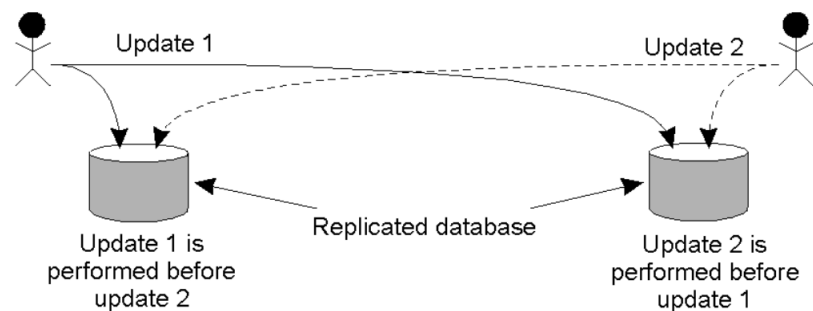
- If $C_i(a) < C_i(b)$ then $T1 < T2$
- If $C_i(a) > C_i(b)$ then $T1 > T2$
- If $C_i(a) = C_i(b)$ and $i < j$ then $T1 < T2$
- If $C_i(a) = C_i(b)$ and $i > j$ then $T1 > T2$

It's clear that in this case, there isn't a semantic, I just need to prioritize the processes to ensure their distinction.

In the example we see that the logical clock is a pair, the second part of is the same for every time for each process.



4.2.1 Totally ordered Multicasting



There are two DB replicated (should be mechanism to keep consistency), there is an update, the application try to update all the replica of the DB. In the other side there is a back office application that compute some percentage on the value of the account and send on the same time messages to all the replica to DB. If we don't have any mechanism to handle concurrency we can end with the possibility to have different value in different replica of DB. One way to do it is to implement this totally ordered multicast and the idea is to use total order Lamport to achieve it.

The idea is that we don't apply immediately the operation on the DB, the DBs have queue in which we store the operation. The operations are not applied until there is an ACK to all the parts. One assumption is that there is not infinite delay, other is the message are delivered in the same order in which they are sent (no overpassing of messages). The message is send multicast to all the other parts and they have to ack when they receive. The operations are applied when all the ack are arrived to the queue.

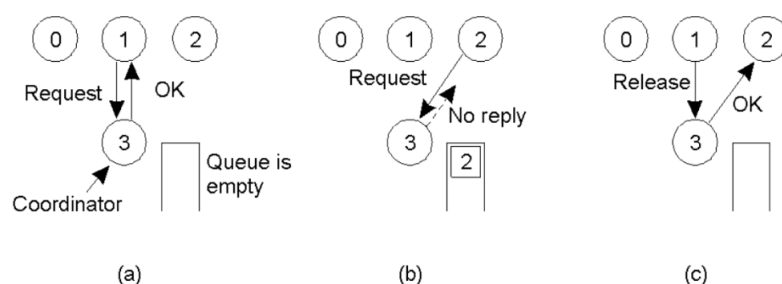
We can observe that after the delay of communication all the queue are the same.

4.3 Mutual exclusion

We have the same problem in one system, there is a problem of concurrency between threads in a kernel that wants to access the same resource.

In distributed system there is not share memory, so they exchange resources but it's not memory.

4.3.1 Centralized algorithm



The trivial idea is to have a coordinator. Here we have 3 processes, the number 3 has the role of coordinator. There is a queue in process 3 and when other processes want to access a resource they have to ask to 3 about it. What's inside the message? the process identifier and the resource identifier, because the coordinator may handle multiple resources.

The coordinator when can give the resource will give it to the process, like in a. In case when he can't give access to resource, he will put the request in a queue like in case b. When the previous process release the resource the coordinator give the access to process 2.

In this case we have a single point of failure, if the coordinator dies what happens here? Could be a process of election of new coordinator. The system could be stuck, because if a process take the resource but never release it this resource is gone. It could be solved by using timeout.

It's more interesting talking about how to do the same thing without a coordinator.

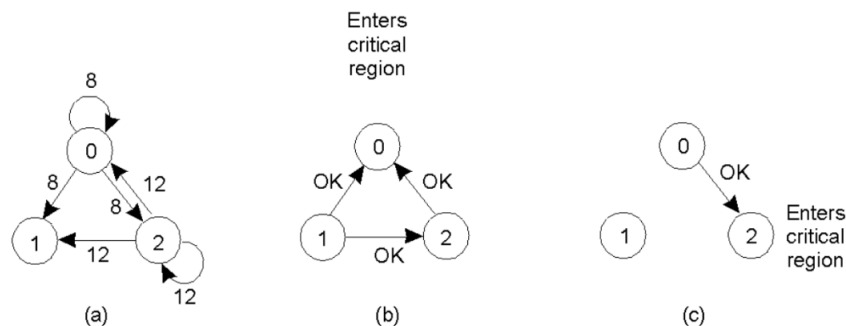
4.3.2 Ricart and Agrawala

There is an assumption: total order (Lamport) of events and reliable message deliver (ack system).

If a process P wants to use a resource R it builds a message containing:

- The name of R
- The ID of P
- The current timestamp: what is the value of my clock

and then this message is sending to everybody (also itself).



In the example we have 3 process. Process 0 and 2 want to access R (specific resource) in the same time. They build the message that contains:

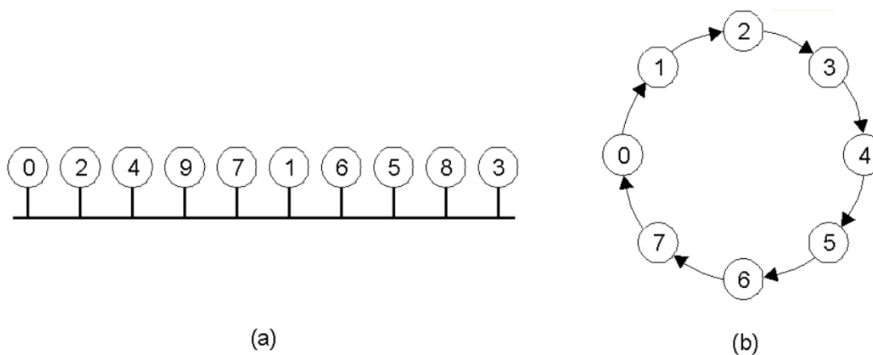
- The process identifier
- The resource identifier
- The clock's value

(I'm p0 I want resource 1 and my clock is saying 8). What happens when someone receive a message? the process 1 is not interested in the resource so will say OK to every request. Process 2 is actually receiving an ok but also receiving a request from process 0 and also itself ask for the same resource, so he compare the timestamp (here is why we need the total order), in this case p2 says that 0 has a lower value of clock so he has priority. So p2 give an OK to p0. P0 receives an OK to P1 and OK from P2 (he also received the request from p2 for the resource R, he also have to remember that P2 asked for resource so he put the request in local queue), now he can actually use resource R. When he's done he sends ok to all of the process that he has in the queue. The process 2 now receive the ok from P1 and because he has received the ok from 1 (don't do the algorithm from scratch) he can access the resource.

Process can go down, fault can happen. We understand that is very difficult to handle fault in distributed system. There is also a performance issue, because broadcasting means sending message to everyone, if there are thousand of process will be thousand of messages.

4.3.3 Ring algorithm

The idea is similar to one of the first local area networks, that is actually token ring. Nowadays we have Ethernet (everyone is sending message whenever he wants). In the previous implementation of protocol in local area networks we connect all the machine in a ring then we exchange a token from machines. When a machine has a token can send a message. The idea here is exactly the same in local are network, when a process has the token can access the resource.



A problem could happens when a node dies, it means that the ring is broken. This is usually mitigate buy let the process knowing not only his successor but also some nodes forward. An other serious problem could be the lost of the token, if a process die when has the token everything is lose. In this case we have to insert back the token in the queue, but be aware if the token is in the queue for delay. This is the worst algorithm if the resource is used very rarely, because nobody want to use the resource and the token is sent anyway, we are sending many messages for nothing.

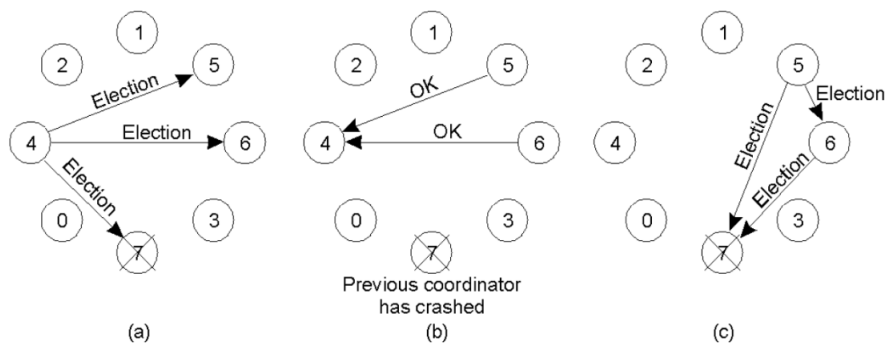
Algorithm	Messages to acquire/release R	Delay to acquire R (num. of messages)	Problems
Centralized	3	2	Crash of coordinator
Distributed	$2(n-1)$	$2(n-1)$	Crash of any process
Distributed as ring	1 to ∞	0 to $n-1$	token loss, crash of a process

4.4 Election algorithms

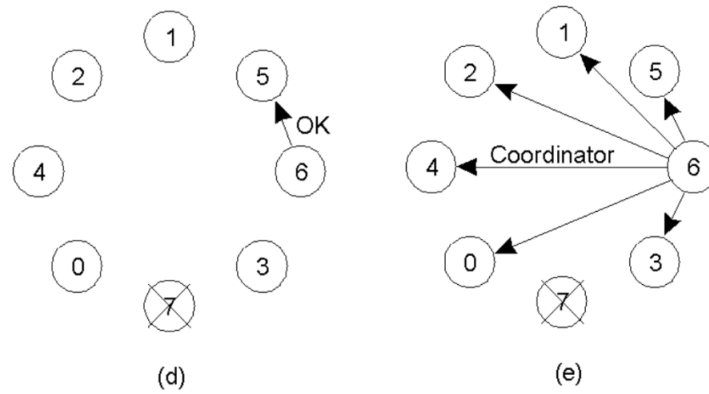
Electing a coordinator process may be useful (for example to implement a centralized solution for mutual exclusion). Election algorithms aim at electing the active process with the highest identifier, this couldn't a good idea because we don't know if the process with high number is active. Also we can use different number respect at the process number, for example the process with the lowest computational load can be elected. Some algorithms assume that each process knows the IDs and can communicate with any other process. However, they cannot know if a process is currently active.

4.4.1 Bully

The idea is that the process with higher ID will be the new coordinator.



Process 4 contacts the old coordinator (7), it understands that it is not active anymore and start an election by sending a message to all processes with an ID greater than their own. Process 5 and 6 answer telling 4 that they will take care. This means for 4 that someone having an ID higher than his that will be the coordinator. Both 5 and 6 start an election doing exactly the same, they send message to process with higher IDs.



Process 6 tells 5 that it will take care of the election. Process 6 wins and tells everybody that he is the new coordinator (it happens with timeout, he didn't receive the ok from 7 because he is down).

What happens if node 7 now will wakes up (he can start a new election)? or if an other election will happens before the previous end? (it works anyway, the same coordinator will be elected), bully algorithm support simultaneous election.

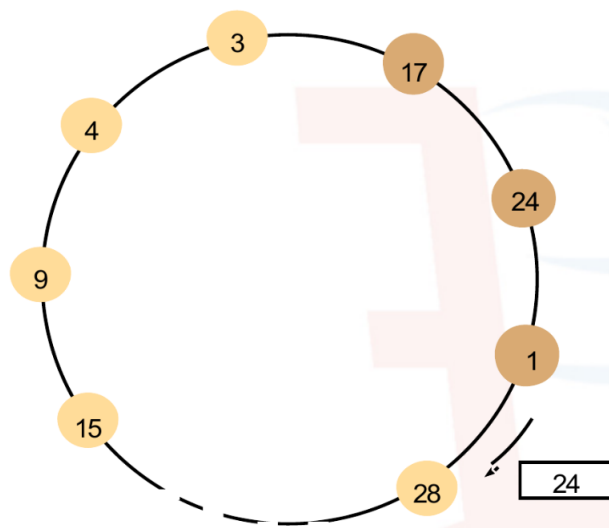
4.4.2 Ring-based election

In the previous example every nodes have to know the address of others.

We can make some assumptions:

- N processes are logically ordered in a ring. Process P_k has a communication channel to $P_{(k+1)MODN}$
- Messages circulate clockwise in the ring without failures
- Processes have unique IDs

The election was started by process 17 because he tried to contact the coordinator that is died. Participant processes are shown in a darker color.



The nodes in ring exchange election message that contains:

- the process id of the current coordinator

When a process receive an election message he has to check if the id contained in the message is lower than his id. In this case he has to put his id in the message. Then he will pass the message to the next node.

While the message return to the node that put his id now he's the coordinator, he will send a message to other (through a ring) saying that he's the coordinator now.

The participant flag is useful because when someone pass the election message flag itself like participant of an election and cannot start a new election or can able to interrupt a new election. When the election end everyone can clean the flag.

Formal steps:

- All processes are marked as non-participant.
- When a process P_k understands that the coordinator is not answering, it starts an election by marking itself as participant and sending to the next node in the ring a message $\langle Election, ID(P_k) \rangle$
- When a process P_m receives $\langle ELECTION, ID(P_k) \rangle$:
 - if $ID(P_k) > ID(P_m)$ P_m forwards the message in the ring and marks itself as participant
 - if $ID(P_k) < ID(P_m)$ if P_m is non-participant it changes to participant and forwards $\langle Election, ID(P_m) \rangle$. If it is already participant it does not forward the message
 - If $k = m$ (the received ID is its own) then it is the coordinator: it marks itself as non-participant and sends $\langle ELECTED, ID(P_m) \rangle$ to the next process
- When a process P_k receives $\langle ELECTED, ID(P_m) \rangle$:
 - it marks itself as non-participant, stores the ID of the coordinator, and unless $k = m$ it forwards the message to the next process.

Failures due to crash of nodes in the ring are handled by each process by storing, not only the address of the next process, but also a few others following it in the ring. If the communication with the next process fails, the message is sent to the first among the ones following it that is active.

The use of the participant/non-participant state helps extinguishing as soon as possible unneeded messages in concurrent elections.

How many messages are exchanged in the worst case (excluding failures)? 3, if the greater ID is the previous node of the one that start the election process we have to do 1 circle to reach the greater id, the second one is to effectively select these process (because the selected node has to see again his message) and the third is to give everyone the info that he is the new coordinator.

Chapter 5

Fault tolerance

Being fault tolerant is strongly related to what are called dependable systems. Dependability implies the following:

- Availability: the probability that the system operates correctly at any given moment
- Reliability: The ability to run correctly for a long interval of time. Up-time in a system
- Safety: failure to operate correctly does not lead to catastrophic failures
- Maintainability: the ability to "easily" repair a failed system

We can have a system that is really available but no reliable, in case we have a system that shut down for few seconds every day.

In the previous lesson we talked about failure of the process, this is the crash failure, that actually is the simple to handle. We have other kind of failures that are a bit different from just a crash:

- Omission failure: the server doesn't respond to the incoming message or fails to receive incoming message or sending it
- Timing failure: a server's response lies outside the specified time interval. Retrying is a possible way to handle this failure.
- Response failure (value failure): A server's response is incorrect. The value of the response is wrong. The server deviates from the correct flow of control.
- Arbitrary failure: A server may produce arbitrary responses at arbitrary times. This is a very critical failure.

Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	A server fails to respond to incoming requests A server fails to receive incoming messages A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure <i>Value failure</i> <i>State transition failure</i>	A server's response is incorrect The value of the response is wrong The server deviates from the correct flow of control
Arbitrary failure	A server may produce arbitrary responses at arbitrary times

Byzantine failure

5.1 Failure Masking by Redundancy

We can do some redundancy for masking failure:

- Information redundancy: for example there are special bits that we can add to our information that can deal with possible noise that can happen during the transmission
- Time redundancy: a transaction is repeated if aborted because of failure of one of its actions. It's not always a good idea, if we do several times the same action we can have a undesired result. NFS
- Physical redundancy: the power supply is the part that usually fail, so we can put extra hw that will be used to recover from failure of some component

In the example we have 3 devices (electronic devices). B need the output from A and C need the output of B, we want to be fault reliant, so that's why we decided to get 3 units of each component. We put a voter for each component (there is redundancy in the voter because they can fault as well) that give the output if the majority agree on something., for example B will get the input from A only if the majority of As will agree. By the way this mechanism of faulting is the base of the paxos algorithm. In this case only one for type can fault and the system will be up and running (we need the majority).

5.1.1 Process resilience

The similar idea instead of having a single process doing something is having a group doing the same thing. Each process receives the messages for the group. In order to do this we need some other middleware software, to joining and leaving the group, broadcasting the messages. In general we may ask how big should be this group? What if the processes need to agree on something (on the command to execute) (means having consensus)?

5.1.2 How much redundancy?

If the type of failure is only crash is sufficient for me that only one survive. So if I have only crash failure to have a k -fault tolerance I need $k + 1$ processes.

I faulty process reply with wrong values, we need that client act like a voter and if the majority agree with the result that'll be correct. So to be k -fault tolerance we need at least $2k + 1$ processes.

If a consensus in the group is needed (to agree on which command to execute) the problem is harder and proved to be impossible in some cases.

5.2 Agreement problems

We can find 3 set of problems:

- Consensus: each process proposes a single value and the non-faulty processes should agree on the same value. We have a group, each process propose a value and they have to agree.

- Byzantine generals problem: one process (commander) proposes a value v . All the correct (non faulty) processes must agree on a value. If the commander is non-faulty they agree on v
- Interactive consistency: each process proposes a value. All the correct processes agree on a vector of values where each value comes from one of the process. Is like having the same queue to grant consistency in DB operations.

5.2.1 Synchronous versus asynchronous systems

If we have asynchronous systems this agreement problem is just impossible. We have to give definition of synchronous and asynchronous systems because there are difference for solving the agreement problems:

- Synchronous systems:
 - Execution on each node is bounded in speed and time
 - Communication links have bounded transmission delay
 - Clock on each node has a bounded drift
- Asynchronous:
 - Execution on each node can occur at arbitrary speed
 - Communication links have different and unbounded transmission delay
 - Clock on each node has an unbounded drift

Actually the real systems are asynchronous, Internet-based heterogeneous distributed systems are inherently asynchronous. Coordination and agreement in asynchronous systems is hard and often impossible. We often make synchronicity assumptions. In order to cooperate in reaching a common goal we need algorithms that achieve a form of synchronization.

The FLP theorem give a formal result to this: When delays in answering messages are arbitrary there is no guaranteed solution to Byzantine agreement (consensus in presence of Byzantine faults) neither to totally ordered multicast, even if a single node fails. There are solutions for partially synchronous systems that can be used to model practical systems and this is what we'll trying to do.

So basically with asynchronous system the Byzantine agreement problem has no solution.

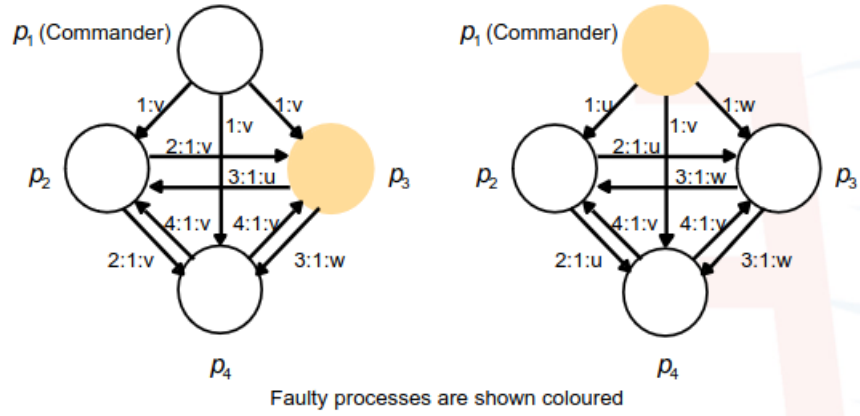
5.2.2 Result on Byzantine agreement with synchronous systems

For k faulty tolerance we need at least $3k + 1$ processes.

Let's see an example of solution for Byzantine generals problem for $N = 4$ and $k = 1$. There is one process that is called commander and the other are called lieutenants. If the commander gives the value v (in the message it says: "I'm the process 1 and I give value v ") to everyone, P2 says to everyone that he's the process 2 and receive value v from 1, the same does P4. The bad guy, P3, can answer arbitrary and says every one

that he received value u/w from P_1 . Here the majority of process agree with the value of commander, so everything is ok.

If the commander is fault we have no chance because the commander value is wrong.



5.2.3 The CAP theorem for asynchronous systems

There are on other formal result, in the asynchronous system we can achieve only two of these properties (here we have an example of reading writing process):

- Consistency: every read receives the most recent write or an error (implies all nodes see the same data)
- Availability: every request receives a response (system is operational at any given time)
- Partition tolerance: the system tolerates an arbitrary number of messages lost (or delayed)

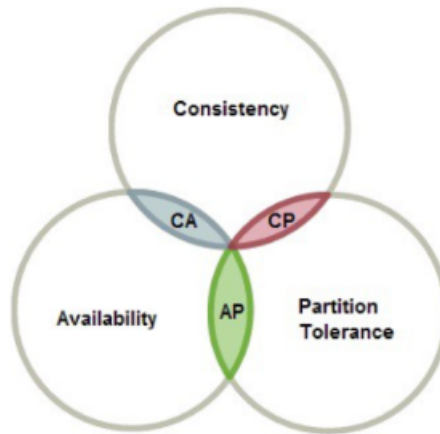
We have to give up something and have a compromise.

There are interception, real systems (the system that we actually use in cloud, dynamo DB relaxes consistency in advantage to availability) relax availability or consistency guarantees (or both):

They don't relax partition tolerance, because this happens, but they coexist with problems of consistency and availability.

5.3 Paxos protocol

The goal of this protocol is solving consensus in a network on unreliable or fallible processors (process) ensuring consistency (it's used in google file system in which the processes need to share info about the system, so they need consistency). The basic version does not cover Byzantine failures, but there is Byzantine Paxos. It tolerates k failing nodes with $N = 2k + 1$. He called this algorithm Paxos because in greece there was a parliament in which the parliamentarians go out to the parliament (going to the beach), like a failure, but they have to take decision.



There are conditions under which it won't make progress but very unlikely to occur. Used by Google in their Chubby (the component that provide the replicas) distributed lock service to keep replicas consistent in case of failure (GFS, BigTable). Also used in Google Spanner (new SQL database, a distribute DB that is an evolution of big table), Apache Cassandra and many other DS.

We don't talk about Paxos.

5.4 Raft protocol

It has the same goal as Paxos, a solution to state machine distribution preserving safety (consistency). It is based on leader election and log replication, we have group of processes and some of them will die unexpectedly and can also occur partition of the group (someone cannot talk with other).

It does not cover Byzantine failures (the nodes trust the elected leader). It tolerates k failing nodes with $N = 2k + 1$.

Raft is a simplest version of Paxos also to be implemented. It's used in kubernetes and Borg in the process that implement the coordination among the clusters.

5.4.1 Leader Election

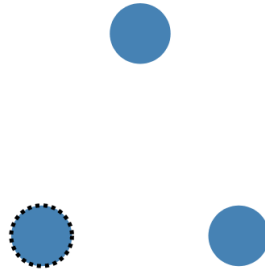
A node can be in 1 of 3 states:

- The Follower
- The Candidate
- The Leader

In the beginning all the nodes start in the Follower state, time has a very important role. There is a notion of term, is like rounds in the algorithm.

If followers don't hear from a leader then they can become a candidate. It's the same situation in which we need a new coordinator:

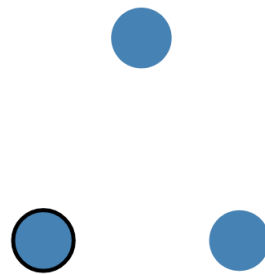
The candidate then requests votes from other nodes. The vote count is null in the beginning.



The other nodes if they have not voted yet in the current turn they will say yes to the node.

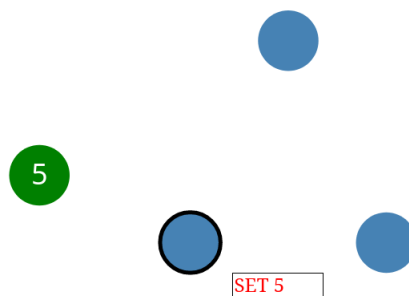
The candidate becomes the leader if it gets votes from the majority of the nodes.

This process is called Leader Election.



5.4.2 Log Replication

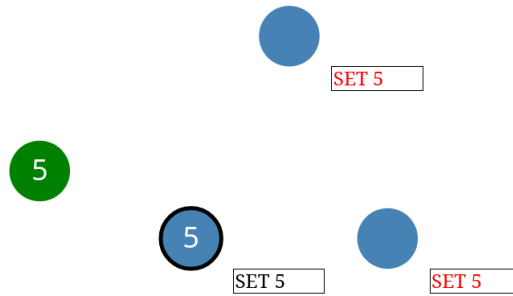
All changes to the system now go through the leader. Each node will have his own representation of the status of the system and they have to agree on the changes, they are trying to share a view on the system. The client will send the new value for the shared data and the leader will tell every other nodes to change that value, but only if they agree. Each change is added as an entry in the node's log:



Each node has a log, this log entry is currently uncommitted so it won't update the node's value.

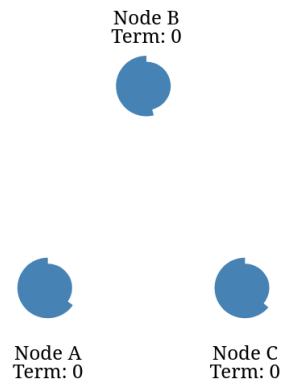
To commit the entry the node first replicates it to the follower nodes:

Then the leader waits until a majority of nodes have written the entry. The entry is now committed on the leader node and the node state is "5". The leader then notifies the followers that the entry is committed.. The cluster has now come to consensus about the system state. This process is called Log Replication.

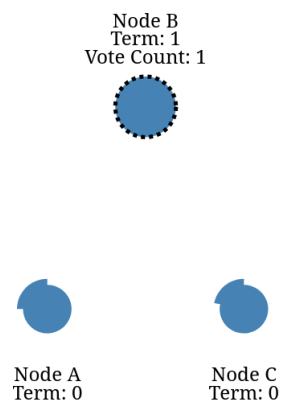


5.4.3 TimeOuts

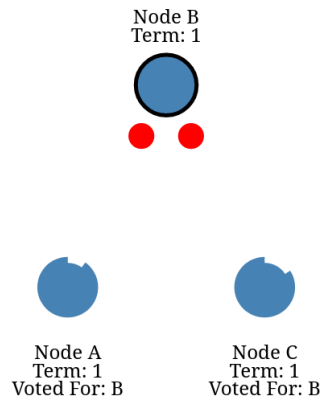
But how long a node have to wait until become a candidate in the election? There are two time out in the protocol which control elections. This time out is not fixed for every node, but is an interval. The first is the election timeout (after this interval if a node doesn't receive a message from a leader become a candidate). The election timeout is randomized to be between 150ms and 300ms.



After the election timeout the follower becomes a candidate and starts a new election term votes for itself and sends out Request Vote messages to other nodes.

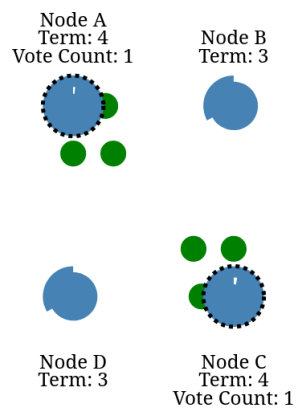


If the receiving node hasn't voted yet in this term then it votes for the candidate and the node resets its election timeout. Once a candidate has a majority of votes it becomes leader. The leader begins sending out Append Entries message to its followers. These messages are sent in intervals specified by the heartbeat timeout (telling that he is alive).

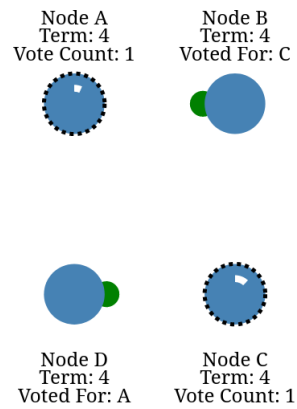


Followers then respond to each Append Entries message and every time they respond they reset the election time out.

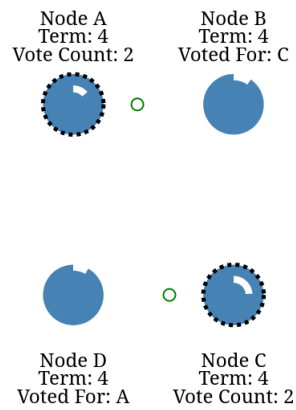
This election term will continue until a follower stops receiving heartbeats and becomes a candidate. A new election process happens if the leader stops. Requiring a majority of votes guarantees that only one leader can be elected per term. If two nodes become candidate at the same time then a split vote is required. Let's take a look at split vote example. Two nodes both start an election for the same term



And each reaches a single follower node before the other:



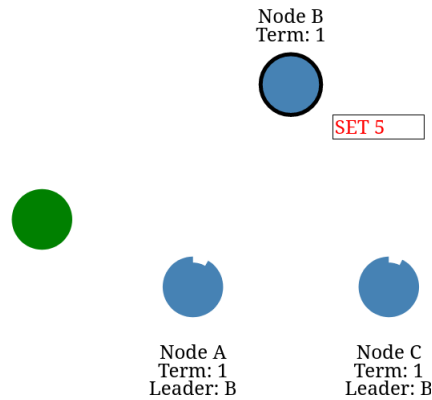
Now each candidate has 2 votes and can receive no more for this term:



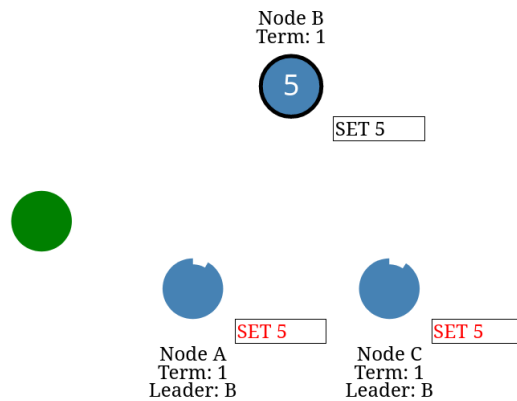
The nodes will wait for a new election and try again. Node C received a majority of votes in term 5 so it becomes leader. Basically if there is no majority they wait for a new election.

5.4.4 Network Partition

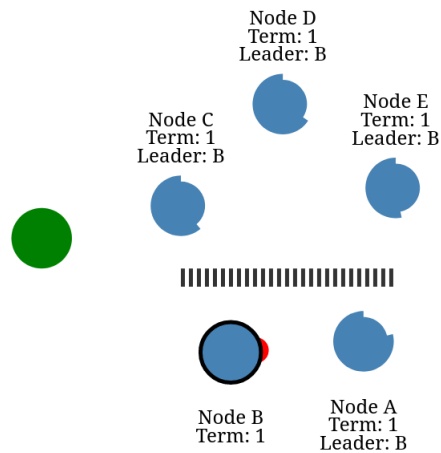
Once we have a leader elected we need to replicate all changes to our system to all nodes. This is done by using the same Append Entries message that was used for heartbeats. Let's walk through the process. First a client sends a change to the leader, and the change is appended to the leader's log:



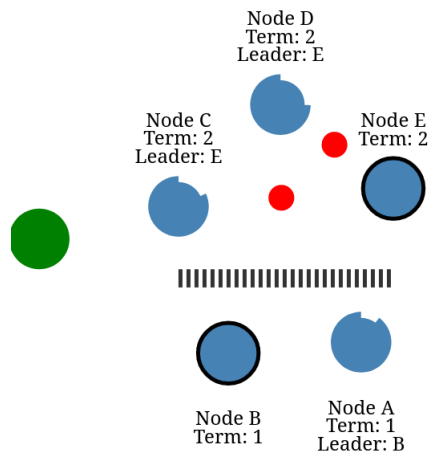
then the change is sent to the followers on the next heartbeat. An entry is committed once a majority of followers acknowledge it and a response is sent to the client.



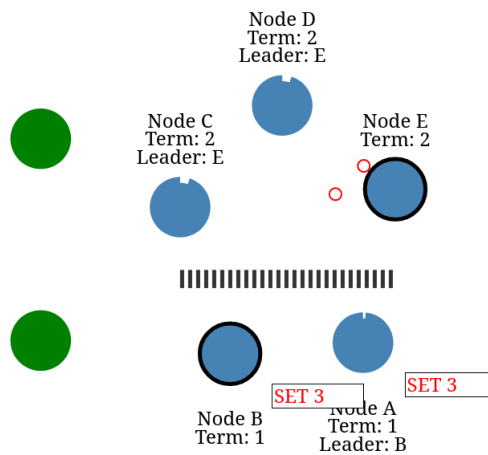
Raft can even stay consistent in the face of network partitions. Let's add a partition to separate A & B from C, D & E.



Because of our partition we now have two leaders in different terms:

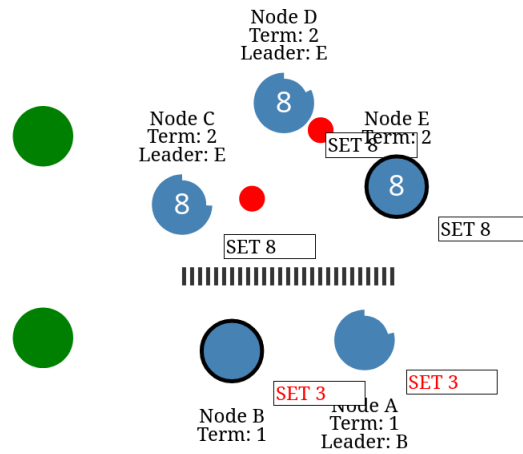


Add on other client and try to update both leader. One client will try to set the value of node B to "3":

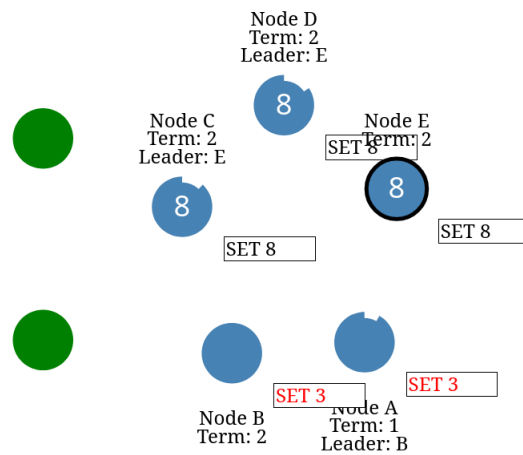


Node B cannot replicate to a majority so its log entry stays uncommitted. The other client will try to set the value of node E to "8", his will succeed because it can replicate

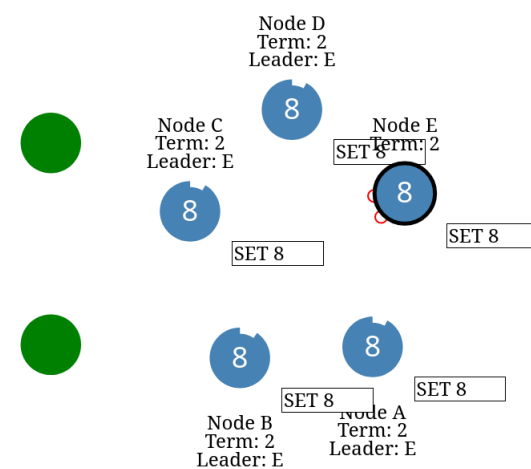
to a majority.: This will succeed because it can replicate to a majority. Now let's heal



the network partition. Node B will see the higher election term and step down:



Both nodes A & B will roll back their uncommitted entries and match the new leader's log. Our log is now consistent across our cluster.



5.5 Other important topics

There are other important topics about failure that we will not talk about, like:

- Reliable client-server communication: Example is RPC in presence of failures
- Reliable group communication: Example is atomic multicast
- Distributed commit: Generalization of atomic multicasts to arbitrary operations performed by members of a group

Chapter 6

Distributed Ledger Technologies and Blockchain

A ledger is a record of set of data that is supposed to be immutable, similarly if we want to know who was the owner of a property we go to a specific office and they have the paper (register) that tells the owner of the property.

So distributed ledger is a ledger that is not cap on a single office. For example Bitcoin (financial) the ledger we can think about a bank for a centralize authority that have one information that has the authority and if we want to do some operation we have to pass through that authority. The idea is that we want to use a distributed system and no authority.

People nowadays are talking about blockchain and web3. Is useful to see blockchain not only for financial transaction but because the idea of distributed ledger can be extended to digital assets that not necessarily are money. Actually in some cases we have pieces of software that we can handle in ledger to be immutable (a piece of code spread in the network and we want to guarantee that everyone can run that).

6.1 Web3

Web3 includes platforms and applications that aim to enable shifts toward a future, decentralized Internet with open standards and protocols while protecting digital-ownership rights. It's not simply cryptocurrency investments, but rather a transformative way to design software for specific purposes. This shift potentially provides users with greater ownership of their data and catalyzes new business models. It's call Web because somehow we are happy with protocol of web (like HTTPS, TCP) so the idea is to use web protocols (also we changed from web1 to web2 when we use web protocol from communication between machines) but the new thing is not only to use web protocols for communications between machine but to include some technologies that we'll see. Mainly cryptography and also if I want to exchange data I would like to know how owns this data, in the original web we ended up concentrating sending all of our data in few big players (like Google, Facebook, Twitter..) so the notion of data ownership has been lost somehow, we get free services (gmail...) and we don't worry about but our data, but we must to worry about. The idea is trying to protect digital ownership something more.

The hype about web3 started in 2021 but is a couple of years that we talking about that approach.

6.2 The DLT System Model

Is a distributed system with:

- Decentralised control
- nodes run by different entities that do not trust each other and may even be malicious (Byzantine behaviour). Runs on web.
- A copy of data records (financial transaction, medial records, sensed data...) is stored at each node. So we have very distributed data, basically we don't put all the data on each node but we use a fingerprint of data that is lighter than complete data.

And of course we have here a consensus problem because nodes need to agree on the history of data (for example a ledger with a history of financial transactions, they need to agree on the history of a property). The reach of consensus in blockchain is NOT about transaction but is about the history (who is the owner of that asset).

The goal in blockchain is to ensure that the majority of nodes agree on the order and content of blocks.

6.3 Blockchain

We actually don't know who came up with this bitcoin idea. The author is this Satoshi Nakamoto that does not exist. He came up with a paper "Bitcoin: A Peer to Peer Electronic Cash System", none before move money on a distributed system. That was a revolutionary paper, not for the algorithms that he used because they were already there (the consensus protocol wasn't new). But made theory practical.

One month before the paper was published the name bitcoin was registered. He took a while before an implementation of this idea come out (2009), there was a first client and people really like the idea.

Few years later (2013) other DLT based cryptocurrencies are created. And here some-one go beyond only cryptocurrency transaction, for example Ethereum is create to handling tokens and contracts.

There are potentially very large set of application that go beyond financial transaction:

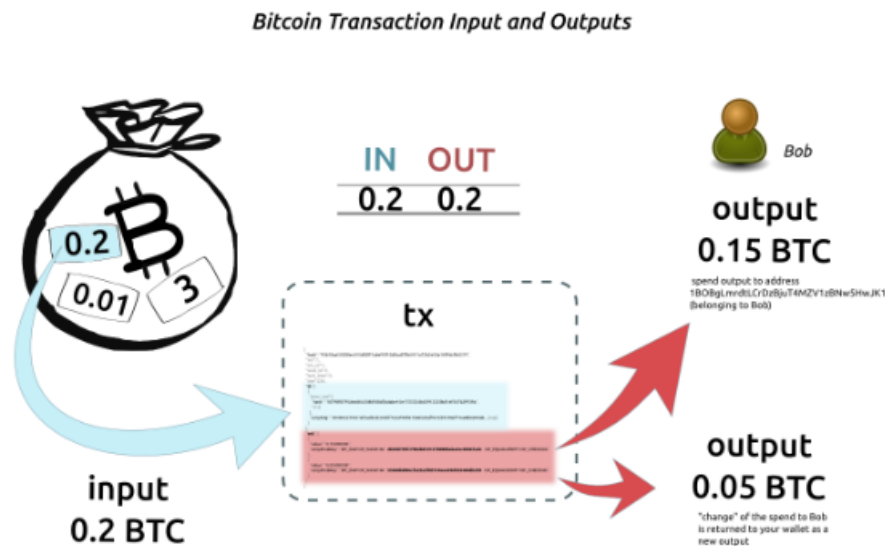
- Food chain
- Medicine transport: to check if the temperature was good

We can use blockchain also for gaming, there are a lot of company trying to go through this business using blockchain for gaming:

	Traditional Games	Blockchain Games
True ownership	Virtual assets are held in the game company server.	Virtual assets are held in the player's digital wallet.
Cooperative global gameplay	Split into regions and servers.	Cooperative global gameplay.
In-game items trading	Trading on third-party websites. Players exposed to frauds and uncertainty.	Trading directly with each other through the use of smart-contract.
Microtransactions	Players must recharge a minimum amount of real money.	Perform directly microtransactions.
Cross-game compatibility	When a player quits the game, he loses each character and every virtual item.	Virtual assets can be transferred between different games.
Frauds	Easy refunds regardless of whether they are faulty.	Transparent and permanent transactions.

6.3.1 Data in Blockchain

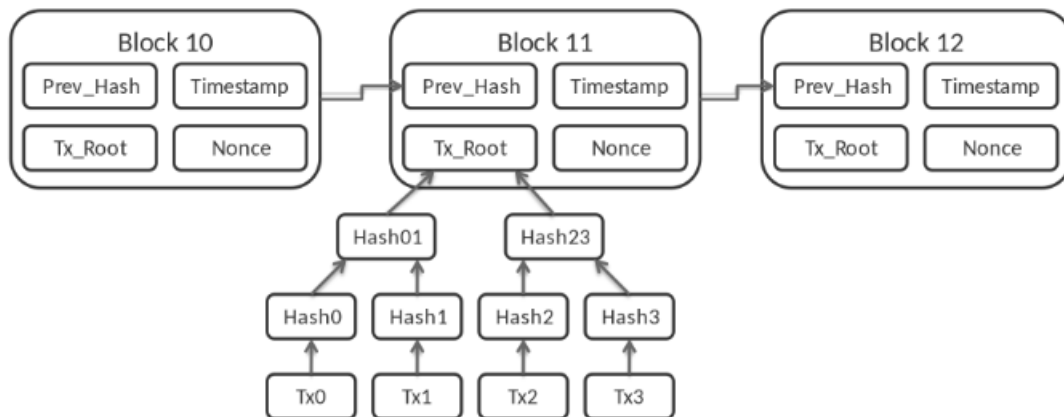
A blockchain is a ledger of transactions. What is a transaction in finance is clear is giving money to someone. In general transaction is data record. In the example works that each participant have a wallet in which they have some assets, in the transaction we want to give 0.15 bitcoin to bob. Also in the ledger is not the balance of our account that is stored but the value of the exchange.



6.3.2 Approach

Each transaction is digitally signed (with private key of the sender) so are used asymmetric cryptography (private and public key) and propagated to all participating nodes. Each node has an identifier, the identifier is the hash of its private key. And there is also a way for the sender to digitally sign his transaction, what it does is sign to the private key, however it also tells to the other its public key that was created together with the private so anybody can check if the transaction was made by Alice using her public key. When a node receives a transaction, it validates it (according to some network-rule) for example in a financial transaction it is possible to check if the sender actually has the money that is spending. This transaction is not considered confirmed yet, we checked if it's ok, but we don't know if everybody agree on this, so it's not committed and that transaction is now pending.

So what we see here is a blockchain, means a set of block, a block is a set of transaction. Usually in the block there are many transaction that are organized in a particular way.



In a block there are time stamp that is important and in general these node will be synchronized.

- There is a value that is called previous hash, is a kind of summary of the previous block, is a signature of the previous block, is a number that change if something in the previous number will change
- There is a Tx_Root that is a transaction route, is a particular data structure called Merkle tree, that is very efficient to tell from the route if certain transaction is there or not
- The 4 thing that is in a block is called Nonce, is just a number and is part of consensus protocol, there is a reward if we do mining, mining means finding this number sooner than everybody else we get rewards

6.3.3 Problems

Since there is unpredictable latency, imprecise clock synchronization, and faulty (malevolent) nodes there are lot of problems to get this thing works:

- The order of arrival of transaction may be different at different nodes
- Some transactions may be contradicting each other (es: double spending bitcoin)
- Different nodes may build different blocks
- Different nodes may end up with different chains (no consensus)

We need to deal with all this tings and clearly we cannot relay on clock synchronization, so its very hard to do all of this with synchronization.

6.3.4 Consensus in blockchain

The challenge is for the nodes to have a consensus on the block and on the sequence of blocks (each node should eventually have the same copy of the chain) Main idea (with proof of work consensus):

- compute the hash of each transaction, and of a block (digital representation of transaction and block)
- Compute the hash of block including the hash of the previous block in the chain
- Include a trick to make the computation of the block hash expensive but very easy to verify. There is not a clever way to do it but we just have to try numbers.
- Make the nodes compete on this computation with a reward, and have the winner propagate its computed block to the others (it is like "electing" a node to impose its block). The idea is that if you cannot really predict who will win, because is kind of random (with some tricks the protocol make every 10 minutes someone get the blocks). The difficult must change because the computational power changes in time. The idea is to make it difficult so people are in very large group of miners. There is a kind of election because who complete the block tells everyone to add that block to the chain, and everyone have to agree that the problem is solved.

Other nodes don't spend times checking if the puzzle is solved. What happens if two block solved the same block almost in the same time? first of all the idea of 10 minute is to reduce the probability to this happens. But actually this is tolerated because in the chain there are forks, this means that each node is not totally sure about the transaction, but after a while only the longer chain will prevails and the "dead" transaction will be put in a pool of transaction that have to be validated again.

6.3.5 Hashing

Let's see in particular the hashing function used in bitcoin, it is the SHA-256 (256 is the length of bits). It is collision resistant (if A is not B then $f(A)$ is not $f(B)$) It is difficult to find A form $f(A)$. And we have quick computation of $f(A)$, easy to verify given A and B if $B = f(A)$.

6.3.6 Examples

Nonce is the number that miner should find, the idea of the puzzle is to change these numbers, so the miners have to change Nonce should start with certain number of zeros in front. I tell you witch kind of hash I like and you need to change the nonce in order to get the hash with that particular property. Note that every small changes to the block require all process to do again.

Block

Block:	# 1
Nonce:	72608
Data:	
Hash:	0000f727854b50bb95c054b39c1fe5c92e5ebcfa4bcb5dc279f56aa96a365e5a

A chain is a chain because of the Prev, that is the hash number of the previous block, is like a reverse pointer. If someone trying to change a block in a chain it invalidate all the chain before, because the hash of a node is based also on the value of Prev, also if he change value transaction inside a block and calculate the correct Nonce again the new hash value is not correct for the next node.

Block:	# 1
Nonce:	11316
Data:	
Prev:	00
Hash:	000015783b764259d382017d91a36d206d0600e2cbb3567748f46a33fe929

Block:	# 2
Nonce:	35230
Data:	
Prev:	000015783b764259d382017d91a36d206d0600e2cbb3567748f46a33fe929
Hash:	000012fa9b916eb9078f8d98a7864e697ae83ed54f5146bd84452cdaafd043

Each node has a chain (different peer), note that if we want to know if every node agree on a chain we have to check the last node because the prev value include all the nodes before. Intuitive is like to say that if the majority of the last nodes have the same hash value they agree on this value.

In a coinbase we don't store on the nodes the balance but only the transaction, if we want to check if someone can do a particular transaction we have to go back in previous nodes looking if this account receive sufficient money:

Block: # 4

Nonce: 18292

Coinbase: \$ 100.00 -> Anders

Tx:

\$ 15.00	From: Jackson	->	Ryan
\$ 5.00	From: Emily	->	Madison
\$ 8.00	From: Sophia	->	Jackson

Prev: 0000df1d632b734f5a5fc126a0f0e8894fb4c8314ba7086b62980559af677

Hash: 0000c694336f88129f3685bd3ba5d67c445dfd8d18bd22f5d87301dd560eb

Mine

Block: # 5

Nonce: 108899

Coinbase: \$ 100.00 -> Sophia

Tx:

\$ 2.00	From: Jackson	->	Alexander
\$ 6.00	From: Ryan	->	Carter
\$ 4.00	From: Ryan	->	Riley
\$ 9.95	From: Grace	->	Katherine

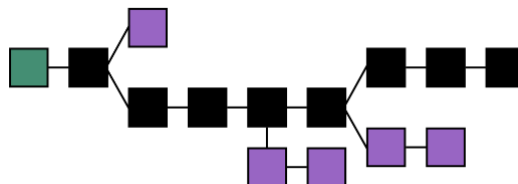
Prev: 0000c694336f88129f3685bd3ba5d67c445dfd8d18bd22f5d87301dd560eb

Hash: 0000fc766a6850dbb98d0cd354c302249ec291df3612ec4dd085105fcfa3a

6.3.7 Proof of Work algorithm

The algorithm used for consensus is called Proof-Of-Work (POW). Verifying a block requires solving a computational puzzle while computing its hash. A miner node needs to find a number (a nonce included in the block data) such that the hash of the block has a particular property. Solving the puzzle requires a brute force approach (it is designed to take a certain amount of time that makes it unlikely that two miners solve their puzzle at the same time) A reward is given to miners that win. The miner don't get the reward just after that have find the number, because there are some things that have to check.

A verified block is sent to all nodes. When a node receives a block, it checks the puzzle solution and then adds it to its local copy of the chain (linked to the one whose hash is in the block).



The miner now try to mine the next block taking transaction from the pool of trans-actions.

All of the nodes are miners? no, because to be a miner we need computational power. And also there is a difference between full nodes (that verify the transaction) and other (that just keep the copy of the chain).

6.3.8 Properties

Assuming most nodes are working on the same chain, the one growing fastest will be the longest and most trustworthy. In order for a malicious node to change a transaction in a intermediate block, it has to re-compute all the subsequent block hashes and prevail over other nodes in the network. Blockchain is safe as long as more than 50% of the work being put in by miners is honest, that's we in byzantine problem the system is safe if 50% of nodes are good.

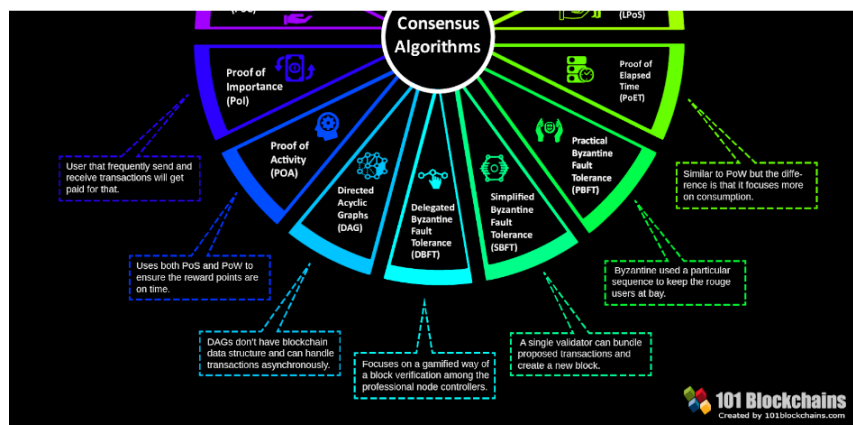
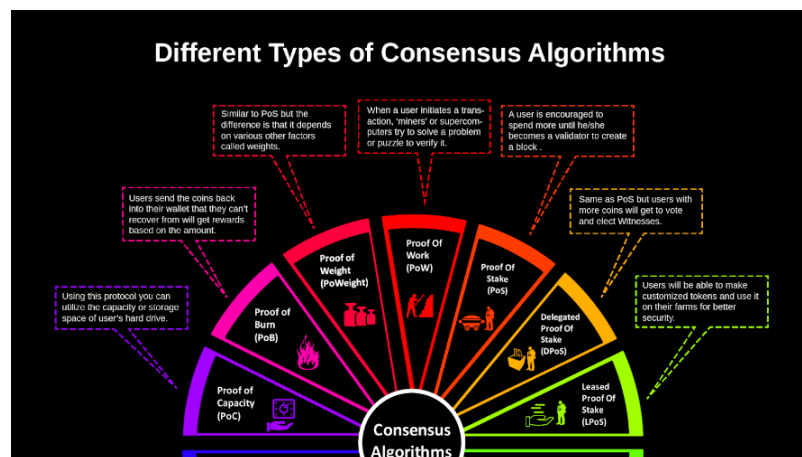
6.3.9 PoW limitation

The major limitation of PoW are computing power consumption and hardware cost. It's estimate that bitcoin consumes a similar amount of power to the Netherlands'.

The other limitation is how much transaction per second we can actually do.

6.3.10 Different types of consensus algorithms

People are actually studying lot of different ways finding consensus. We mention the proof of stake, ethereum switch to this other algorithm, the idea basically means that if we have a lot of power in blockchain we decide which block come next. There is no puzzle to solve but the idea of hashing is the same. But how decide which is the next node and how they agree? there is some randomness on who can propose the block, we are using the term propose because there are some nodes that are validators that check what the guy is proposing. The idea is that if we have a lot of money in this system we don't have interest in braking it. If you have in stake in there you can be a candidate (like being a candidate in an election) and use the wallet like a guarantee (if i do something wrong take my money). Once my block proposed is validate it have to be spread around to other nodes. It at least solve the problem of electricity. In addition to ethereum there is also Dash that use this process.



6.4 Smart Contracts

This idea was introduced in etherium blockchain and is having a lot of success. We can use blockchain for more than financial transaction, smart contract is piece of software, each blockchain proposing its language (etherium's language is solidity). The idea is kind of regulating the transaction between different parts. The example is like a crowdfunding platform without the crowdfunding platform, while we do crowdfunding we put perks for which give us money. We can do in decentralized way.

The idea is that we have some contract we can put in blockchain, put software in blockchain means that this software is replicated and immutable. Each node can verify if the conditions of the contract are satisfied and perform the consequent actions (the output is "distributively validated"). In the end all nodes should agree on the resulting "state".

6.5 Bitcoin and Ethereum are permissionless

We can enter in the blockchain tokens, a transaction is an example of a token, money is an example of the token. So these are called fungible tokens, NFT stays for non fungible tokens. Fungible means that we can use tokens and exchange them. An idea of NFT is to take grant of our data, certified, and grant access to our data, the idea is if everyone have control of its data the things can change, if we collect the data and put in a distributed file system and put a fingerprint of data in blockchain we can sell our data, when the data is out I want a license that these are my data the license is a NFT. The idea is that our data can be digital assets and smart contract can automatically handle the date. Ocean protocol is already proposing an infrastructure to do this.

There are 2 main type of DL:

- Permissionless: are to one that we have in mind and we've talking, means that we don't need the permission, are decentralized, don't track transaction, unconditioned access to the ledger (we can look through the ledger), transactions validation and new "coins" generation by miners, pseudo-anonymity of participants, immutable transactions.
- Permissioned:
 - Decentralised DL that tracks all transaction
 - One or more trusted third parties: no pure peer to peer architecture
 - Conditioned access to the ledger, for example a company want that only partner can access to the ledger
 - Transactions validation and new "coins" generation by miners
 - Known identity of participants
 - Immutable transactions

What is the difference between "permissionless" and "permissioned" DTL's? The latter assume at least one trusted node and enforce access control to the DLT.

6.5.1 Hyperledger Fabric

It's made by consortium of industries to develop open-source blockchain technology for business, hosted by the Linux foundation.

Is a permissioned DLT. It's a modular architecture adapting to different requirements: private transaction, confidential contracts, different consensus protocol (raft). The distributed apps (contracts) are build in general-purpose programming languages, this is very important because here we can use general-purpose programming language. There is no dependency on a native cryptocurrency, while in Ethereum there is a specific cryptocurrency.

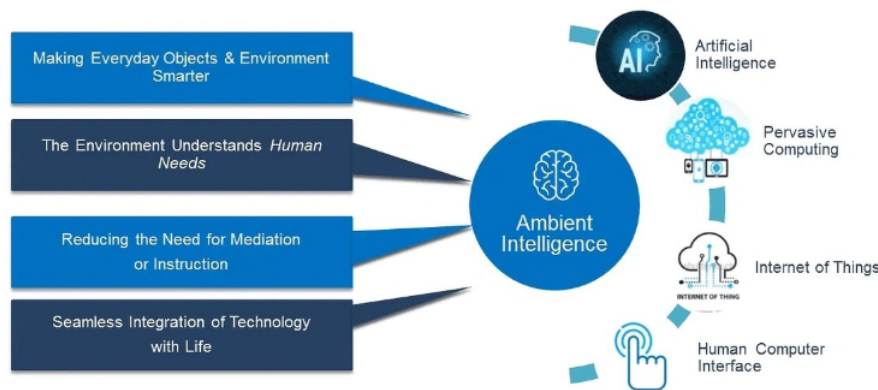
Chapter 7

Pervasive Computing

This pervasing computing is an enabling framework for what we call ambient intelligence. The idea is that in pervasive computing we consider nodes for distributed system also unconventional nodes (laptop, smarthphone...), in general device that have computation ability and connection ability.

If the enviornment has to understand the human needs means that the signal coming from sensors need to be analyzed, this don't make sense for a single value (temperature) but make sense if I can correlate data from different sensors in order to understand what is happening in a house, understand the context.

There are a lot of AI in analyzing in data, but remember that AI is not only machine learning.



The pervasive therm is used as synonymous as ubiquitous. The vision of pervasive computing is that the technology will disappear but the environment will react to the user. "The most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it". It's related also to cloud computing technology.

7.1 Distributed Pervasive Systems

Definition: it's a distributed system with the following main features:

- It includes unconventional nodes: objects with computing and communication capabilities (smartphones, smart appliances, smart meters, sensor networks, ...)
- Adaptivity: the system logic considers the current context and adapts the system behavior for optimizing the system goal

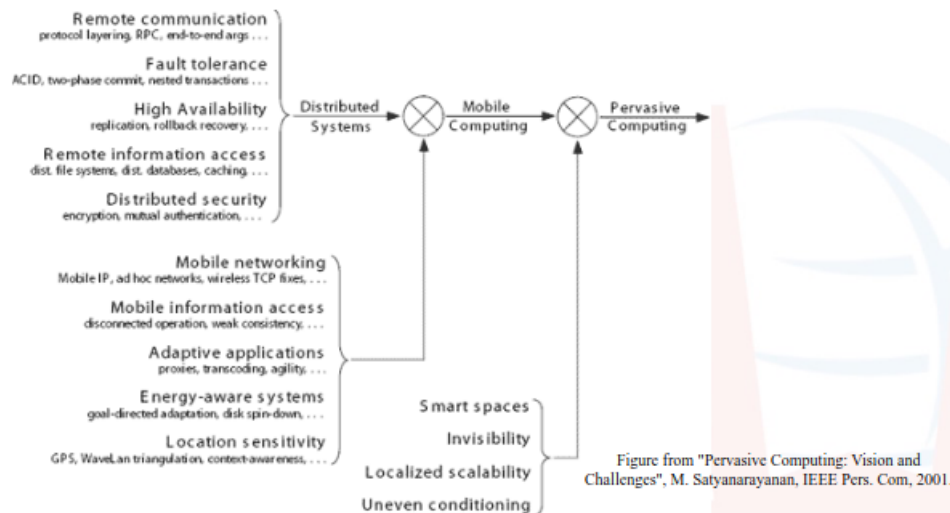
The context is the current situation when we consider a service to be provided. In general we ask for something and the system not only look for our question but consider also the context (where we are...), it has to provide us what we want without that we say everything.

This kind of systems exhibit high volatility:

- failures of devices and communication systems
- changes in the characteristics of communication such as bandwidth

- creation and destruction of associations between software components resident on the devices: the IOT device have to join local network and this association is an issue

The problem that we have with distributed system multiply when we have smart ambient.



7.2 Mobile computing

We have several issues in mobile:

- limited resources: energy, CPU, memory. Drains the battery very quickly is a problem.
- different types of interfaces: today we have touch screen
- high variance in connectivity
- variable location

Main research topic:

- Networking
- Mobile information access
- Mobile data management: location is a sensitive data, we don't want to be continuously trace
- Positioning: indoor and outdoor localization, proximity, tracking
- Software: app and mobile service design, development and testing scalability

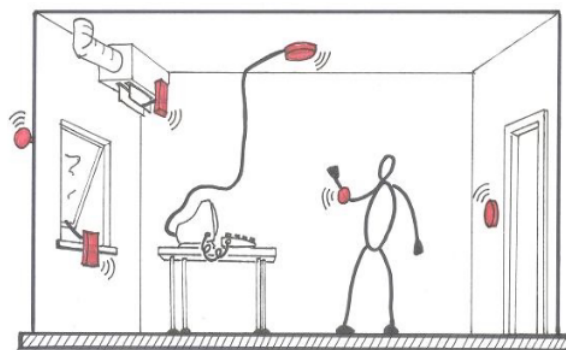
7.3 Pervasive Computing

From mobile nodes to embedded computers and affordable Internet connected sensors and actuators, listening and affecting the physical world. The idea is that we'll have more connected wearable nodes like:

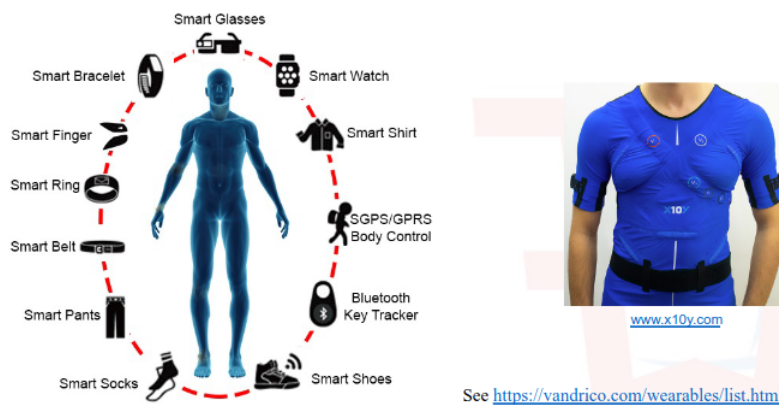
- Oura ring: has a lot of sensor and can provide a lot of useful information
- Google glass, meta glass

7.3.1 Sensors And Actuators

There are some devices that are sensing data and some other that are doing some works.



Sensors and actuator are different thing. In general sensors (and actuators) can be embedded in devices with limited computing and communication capabilities. Sensors can be embedded in personal smart objects. Sensors/device can be interconnected in sensor network, in some case we need more lightweight protocol than Internet, we cannot keep the WiFi antenna on all time. There are tons of wearable sensors:



7.3.2 Sensor Network

One of theoretical problem is that if there are smart ways to collect data? the dumb way is that all the nodes give the data to the gateway. The main goal usually is energy saving. There are questions concerning sensor network:

- How do we (dynamically) set up an efficient tree in a sensor network?
- How does aggregation of results take place? Can it be controlled?
- What happens when network links fail?

7.3.3 Smart appliances

Nowadays WiFi chip are cheap so industries put connection capability into everything, but this doesn't mean that they are smart because these systems are close. They are usefully but not here.

7.3.4 Domotics

He give the example of sensing floor. One problem in this kind of system is to understand who is doing what, understanding who is generating an event is difficult.

Domotics is the smart home case, the trivial application of domotics is that I'm coming home so heat up the house or turn the light on.

Health and energy are top application for this kind of system. Optimizing energy from whole city can do a difference.

We have state of device and some rules, if you observe this please do this action.

7.3.5 Problems

More and more smart devices are available but they are using different network standards and interfaces. We are still looking for what is the best standard for the network, is the same like we did with local area network. Is not the case for platform as well. Data flows to proprietary clouds with control and visualization available only through proprietary cloud-based apps. We have no possibility to access directly the data coming from sensors, usually sensors (like smartwatch) sends the data to the company that sell the sensors and only after we can have access to API that give us back the data, but no row data (idea of smart contract for data).

Challenges:

- Difficult to integrate data and build automation across multiple devices
- Difficult to home owners to control devices through multiple apps: there are company that are trying to provide universal app
- Difficult to protect personal data

But if we consider open source possibility these are the emerging platform. The goals are:

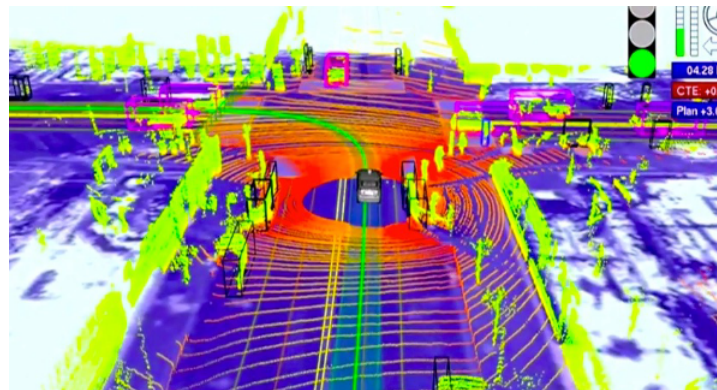
- Supporting and integrating control for multi-brand and multi-network devices
- Enabling local data management: this is getting really difficult. Means that i don't want to give access to my home to somebody outside of my network, this is almost impossible. I'd like to run locally these platform and collect data at home, it's very difficult in particular to integrate the vocal assistant.

7.3.6 Home Assistant

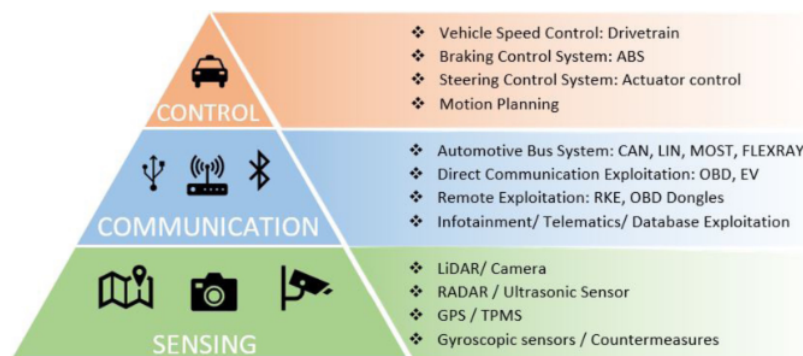
- Open source project in Python
- Local integration and control of multi-vendor multi-network devices in smart-homes
- Home automation
- Graphical UI
- Easily integrated with DBMS and cloud

7.4 Smart Vehicles

The image is the representation of what the car see, it receives a lot of data:



This is by itself the all domain, we are going in this direction:



7.5 Why "smart"

The notion that we have to have about smart device is not the one the vendors has, that basically is that has the WiFi chip. But they:

- Running algorithms (locally or remotely) to analyse data and understand "context" by exploiting AI techniques

- Offering personalized context-aware services: means react (according to the global goal of the system) to the situation considering the current context. Personalized exercise based on my age and sensors data.

7.6 Smart environments

- Smart Home:
 - automation/security services
 - assisted living
 - energy management
- Smart Building: utility management (energy, water...)
- Smart Vehicles:
 - safety
 - energy optimization
- Smart city:
 - Traffic/transportation/air quality. Crowd sourcing to collect data and improve the city, for example communication company know how much phones are on areas
 - utility optimization
 - safety
 - accessibility
 - tourist services
- Smart country
- Smart world...

7.7 Pervasive computing

New issue:

- smart spaces need to be effectively used (adaptiveness, context-awareness, trying to anticipate the needs of people using the service)
- invisibility: interaction with users should be minimized (system transparency)
- resources in a pervasive environment should be discoverable and dynamic association/collaboration should be enabled. We are still bit far from this.

7.8 Current research topics in pervasive computing

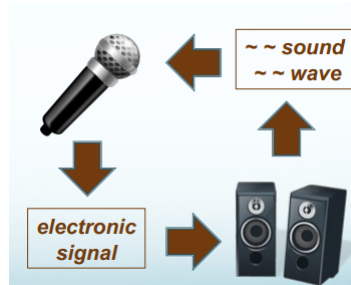
- Context and Activity Modeling and Recognition:
 - how to represent rich context and how to reason with context
 - how to recognize human activities from streams of sensor data, ...
- Crowd sensing: distributed acquisition of data from users (from their smartphone, their car, their home, ...)
- Energy analytics: analysis of smart-meter data and enabled service
- Pervasive health: wearable devices and environmental sensors enable continuous monitoring. Goals:
 - early detection of health problems
 - monitoring health conditions in cronic diseases
 - facilitating rehabilitation at home
 - extended independent living (reminders, alarms, intervention, ...)
- Pervasive transportation:
 - road sensors, car sensors, driver smartphones coupled with inter-vehicle communication enable new services
 - Goal: mobility optimization, emission reduction, hazard detection
- Edge Computing: when we need real time system we need real time reaction to the data that is coming from sensors, we need computation that is close to where data are collected, close to actuation. Move computation closer to the data source.
- Security and privacy:
 - IOT devices are currently an easy target
 - A huge amount of sensitive information is stored in the cloud

Chapter 8

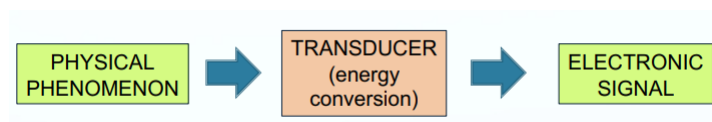
Sensor Data Management

8.1 Sensor Devices

Sensors and actuators are part of the same family that are called transducer. For example voice is a physical phenomenon that is captured by microphone (sensor) that transform the voice in an electronic signal that is transmitted to the speakers (actuator).



Every sensors detect physical changes and produce an electrical signal. Some sensors work continuously in time, and they transform this phenomenon in electrical signal and produce data stream, we have to take that data and do stuff.



We have many types of sensors, physical:

- Motion sensors: they measure acceleration forces and rotational forces along three axes. Accelerometers, gravity sensors, gyroscopes, and rotational vector sensors
- Environmental sensors: they measure environmental parameters, such as ambient air temperature and pressure, illumination, and humidity. Barometers, photometers, and thermometers
- Position sensors: they measure the physical position of a device. This category includes orientation sensors and magnetometers.

Some environmental devices can be used in some field, for example agriculture, for monitoring stuff.

We have already talk about smart home and wearable to get data (also implantable).

8.1.1 BLE beacons

The BLE beacons are devices that only send out Bluetooth signal, the idea is to build application like we go to a store and getting closer to a product, our phone is getting signal from beacon and we'll receive an advertisement . But Bluetooth has several problems, the biggest is interference.



8.1.2 Sensing device components

We'll see the architecture of device, because the sensor is one small part of device. There is:

- Processing subsystem: local data processing
- Wireless communication subsystem (could be wireless or not, to be a node of distributed system)
- Power source (battery, solar). Usually battery is what determine the size
- Local data storage: you may be produce data more than what we can actually send out, we need at least some buffering ability
- Wired interface: because if we want to do firmware update or maintenance
- Mobilizer: change location or configuration

SENSING SUBSYSTEM (data acquisition from the environment)	PROCESSING SUBSYSTEM (local data processing)
WIRELESS COMMUNICATION SUBSYSTEM	POWER SOURCE (battery, solar)
LOCAL DATA STORAGE (caching)	WIRED INTERFACE (miniUSB...)
MOBILIZER (change location or configuration)	

8.1.3 Virtual sensors

There are also virtual sensors, as computer scientist we can design application using virtual sensor, there are some services that are offered and we can reach sensors that are not our properties. There are other service that are not really sensors but we consider like virtual sensors, for example Google Places API revealing semantic location (we send coordinates and it returns semantic location, for example we are near supermarket)

8.1.4 Smartphone-based sensing

Smartphone-based sensing, an app that collect data to infer:

- Activity
- Disposition
- habits
- Surrounding

8.1.5 Actuators

Actuators are also a particular type of transducers, they are usually powered. They convert energy into an action (motion, switch), upon receiving a command from a control system. Examples: dim lights, open a door, steer a wheel, open a radiator valve...

Can an IoT device include both sensors and actuators? Yes, even if it is battery powered

8.2 Networking for IoT

There is not a standard, we'll see some technologies:

Technology	Range (typical/max)	Max data rate	Power consumpt.	Frequency range (GHz)	Topology, Max nodes	Main Apps
Zigbee	10/20 m	20/ 40/ 250 kbps	Low	2.4	star/mesh, 65k	HomeAut, smartGrid, RC
Z-Wave	10/30 m	9.6/40/100 kbps	Low	0.868 EU, 0.908 US	mesh, 256	HomeAut, Security
Bluetooth Low Energy (BLE)	6/100 m	1-2 Mbps	Low	2.4	scatternet	Audio, persDevice, healthcare, beacons
WiFi (802.11n/ac)	30/200 m	100-1000 Mbps	High	2.4/5	star	LAN

8.2.1 Matter

The effort to find a standard is also in the higher level (in networking stack), in the application layer we have Matter, that is an open-source standard for smart-home device connectivity. It's promoted by many industries (Apple, Google, Samsung, Amazon...).

It's focused on interoperability, security, easy-of-use, reliability. In the first release it supports WiFi, Thread and Ethernet protocols. Designed to run on any network stacks that support IP. In the Matter 1st release, it supports WiFi, Thread, and Ethernet protocols.

Technology	Range (typical/max)	Max data rate	Power consumpt.	Frequency range (GhZ)	Topology, Max nodes	Main Apps
Thread	10/20 m	20/40/.../250	Low	2.4	Mesh, IPv6 addressable, 256+	HomeAut, IOT
ESP WiFi Mesh	Wide with 10/200m between 2 nodes	10 Mbps	Medium	2.4	Mesh/Tree, 1000nodes (in layers with 10/node)	HomeAut, IOT
5G (LTE-M, NB-IoT)	wide	LTE-M:Mbps, NB-IoT: kbps	Medium/Low	Eu 3.6	cellular	Mobility, stationary IOT
LoRaWAN	wide	very low (20-50 kbps)	Low	EU 0.863 - 0.870	star	utility metering

8.2.2 Gateway

They collect data from distributed sensing and actuating devices. They have antenna to communicate with sensing devices that participating in the network. Some of them has DBMS on board.

8.2.3 Platforms

To build applications that have to deployed is better find platform that works for:

- Device management
- Device control
- Data acquisition
- Data processing
- Data visualization

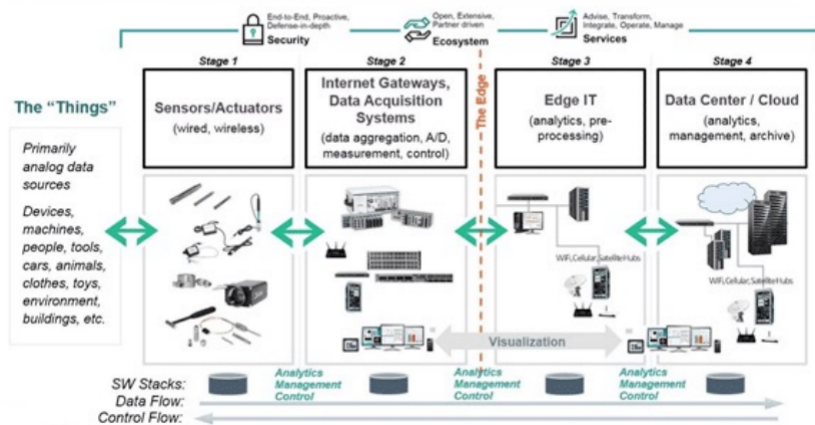
The idea is to delegate something to other.

8.2.4 Discovery and pairing

Smart devices can frequently appear and disappear in smart space, and this is a big issue.

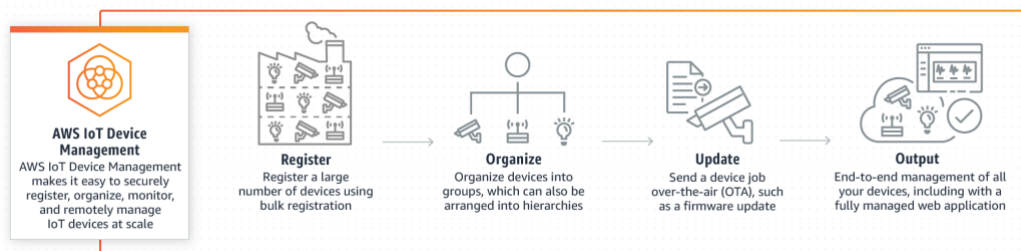
Network bootstrapping is register the device to the network and assign a unique address, we solved the problem with WiFi using DHCP. Association is associate the device to the application based on the smart space boundary and device service (nowadays we have to do some physical thing to the device, for example push a button n time).

The 4 Stage IoT Solutions Architecture



8.2.5 Device management

There are some platform (Amazon) that solve the problem of device management when we have large scale application. Manage means firmware updates very often, and have to do over the air. We would like to register a large number of device using bulk registration, not one by one.



8.3 Managing and querying sensor data

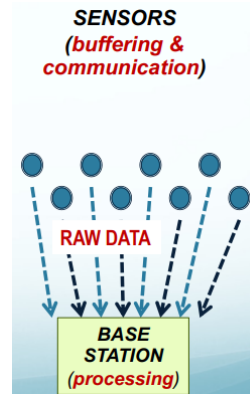
8.3.1 Data acquisition

The scenario is that we have sensors that produce data and we have to collect using some methods. The most basic method is BATCH processing, in which we have device that have buffer ability, than we send the data to base station (gateway). Properties:

- No processing on the sensor
- The raw data come to the station, the base station receives all the data (but with a delay)

In this case we have different problem respect traditional DB:

- Continuous vs one-time query: continuous query is for example know how much people are in the building and we want this information continuously



- Strong spatio-temporal characterization of data

The second basic method is SAMPLING it consist in not sending all the acquired data. The difference between Batch and Sampling is that the second one doesn't send all the data. We send some data representing what we want to keep because, for example, we read data only at a given frequency. Why to do that is for energy, that is the most important thing that we want to to optimize.

SLIDING WINDOWS is an other way, instead of sampling that exclude some values we want to use all values but not send all of them. We need some aggregation function for send data. The idea is to have a group of value and send a value for the group, the window decide how much values in group or maybe in therms of time (every second i send the medium). Here we hve run-time computation on the device. The base station receives a continuous stream of approximated sensor reading (with a small delay).

Here we see an example from accelerometere, in some cases we send values not only of one function, but from several function:

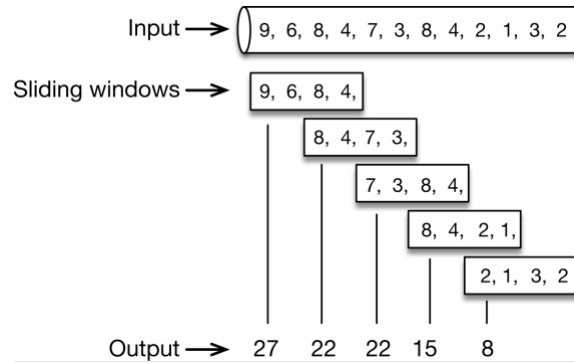
T=0.0	T=0.25	T=0.5	T=0.75	T=1.0	T=1.25	T=1.5	...
Acc-X	Acc-X	Acc-X	Acc-X	Acc-X	Acc-X	Acc-X	...
Acc-Y	Acc-Y	Acc-Y	Acc-Y	Acc-Y	Acc-Y	Acc-Y	...
Acc-Z	Acc-Z	Acc-Z	Acc-Z	Acc-Z	Acc-Z	Acc-Z	...

Features at first second:

- Max/Min/Avg Acc-X,Y,Z, Total-Acc
- Max/Min/Avg Max/Min/Avg Tilt
- Skewness, Kurtosis, Fourier coefficients, ...

Approach used in
accelerometer-
based activity
recognition

In this case we see consecutive windows, there is a overlapping-0 because the next windows start when the previous end. What we usually do is considering overlapping windows. So we have to choose also the overlapping factor (also than the dimension). In this case the quality of the data increases, because if something happens in the middle of the windows in the contiguous windows we are not able to catch that event. But in overlapping sliding windows is possible to catch that type of event.



8.3.2 Advance methods

Objectives:

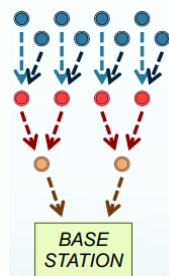
- Save energy
- Improve data quality

Transmission is much more power consuming than processing. The cost of sensing depends on the sensed phenomena.

So we should focus on transmission. They are trying some algorithmic way to decrease the transmission consumption.

In-network query processing

One is In-network query processing, if i want to compute the average temperature in the building and have a lot of sensors instead of having all the sensor communicating to the gateway why we don't let communicate to them self and communicate to the gateway only the result? it's the opposite to centralized processing. We have to build an overlay network and then we perform data aggregation in intermediate nodes to reduce transmission, to compute average, min/max... In the end we have the medium overall but we have distributed computation.



Duty cycling

The radio is put on sleep mode most of the time (radio because is the most expensive, in term of energy consumption). There are scheduled algorithm for coordinate sleep/wake up scheduling. The idea is that the antenna is always on sleep mode and periodically is turned in order to check if there are some messages. The problem is if we have urgent

messages to be delivered and the antenna is off. Usually we put a threshold in some value, that wake up the radio if it was overcome.

Mobility-based

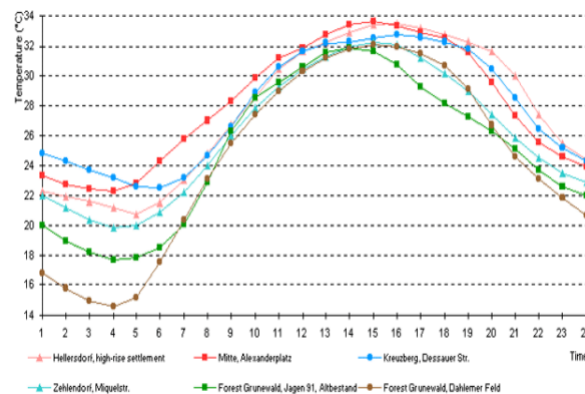
The base station may be mobile because nodes may be carried by people, cars, animals.. We have to exploit mobility (when available) to better distribute the transmission cost between nodes. Communicate when they are close to the gateway.

Model-based approaches

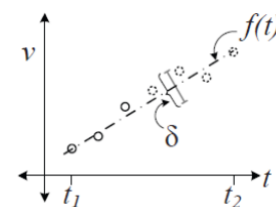
Which are clever way to save energy.

It's statistical model. Usually when we deploy sensors around we have an idea of a distribution of values that we want to obtain, somehow we know if we look the historical value the value that we want in time t . This is the model, we approximate the distribution with a function.

Sampled data may have strong spatio-temporal correlation. The objective is to reduce the number of samples (red or transmitted) while keeping good data quality. The idea is that sensor don't tell if we have the expected value, if nothing strange happens no communication at all (maybe some I'm alive message), if values follow the distribution save energy.

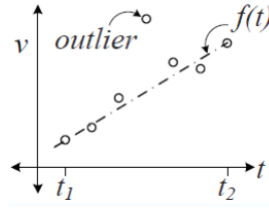


This is the model-based for data acquisition, the goal is efficiently acquiring samples from the sensors. In the figure we see the function that approximate the history of data, we have also data that are close to the expected value (closed if they are in the δ distance). The goal is efficiently acquiring samples from the sensors.

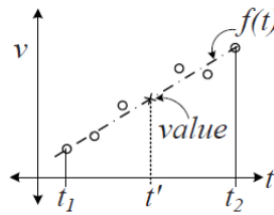


But we can use model-base approach for several things. One is data cleaning, I should find way to know that some values are strange and I can delete them, these value are called outlier. This is tricky, I can say if i'm inside delta do nothing, but if i'm outside i

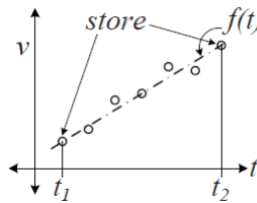
cannot delete without reasoning, probably we can have an other threshold to know if we are really far away.



Query processing, if I have to give a value at time t , and the sensor is not storing the data (because are similar to the model) we give the value from the model. The goal is processing queries by accessing or generating minimal amount of data.



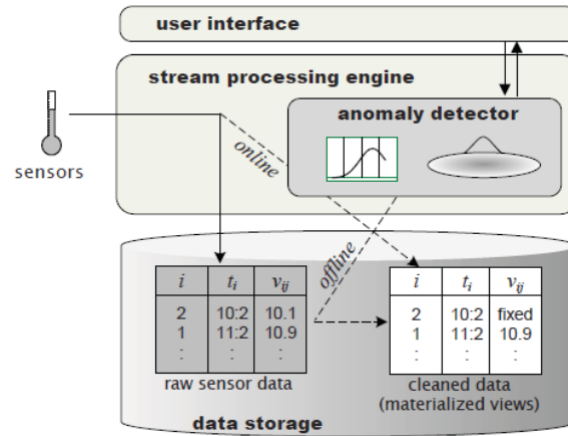
Data compression, we continuously get data and we have to optimize storage, is like optimizing DB of stream of data. Between t_1 and t_2 data follow this function. We give start and end value and in between they follow the function. Clearly the model may change in time. The goal is eliminating redundancy.



8.3.3 Sensor data cleaning

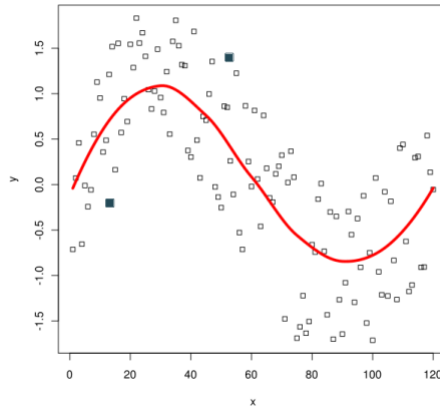
Typically sensor data cleaning is done in the base station, because we can compare the value from different sensor that are near, if they give completely different value something goes wrong.

Typically we have software model that do data cleaning, trying to detect outlier. They use some models in order to know the normal behaviour, the most probable sensor values are inferred using statistical models. And other strategies (average, comparison with other sensors).



Here in the data storage we also want to store dirty data (raw data), because we never know, maybe we see that we did the cleaning badly. This data cleaning can be done online or offline, clearly if we do offline we need to store raw data.

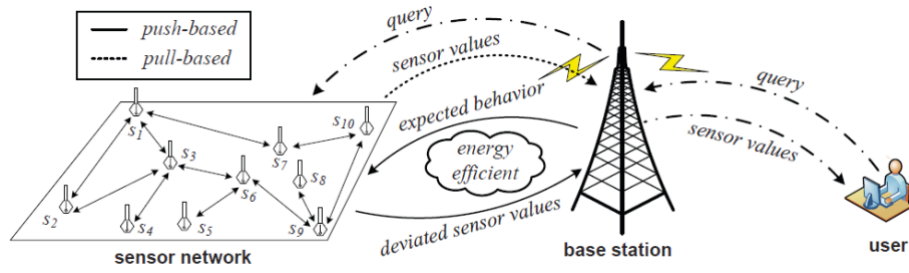
We can use different kind of functions, it's not trivial to come up with proper model. Using regression model we can capture both continuity and correlation.



8.3.4 Data acquisition

In this case we see push vs pull:

- Pull: every 10 seconds I want the data, so I make a query and the sensors give us values
- Push: is using the model base. The base station communicate the expected behaviour, because we have the model. The answer from the devices is only deviated sensor values

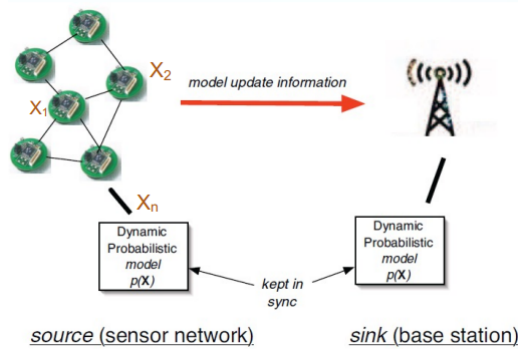


Pull-based data acquisition

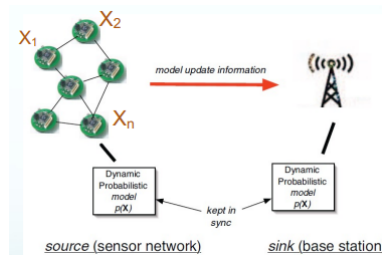
Is like SQL like query:

```
SELECT sj, vij FROM sensor_values SAMPLE INTERVAL 1s FROM 10s
```

We just asking to the sensors to give value at the cert frequency maybe applying sliding windows or function. And we constantly update our table. We can use linear model, linear prediction model that considers only temporal features and treats each sensor independently



Or markovian model, this is more complex. In the previous model we haven't dependency on value in prediction model, we can consider the value of the other sensors. It consider complex dependencies among sensors.

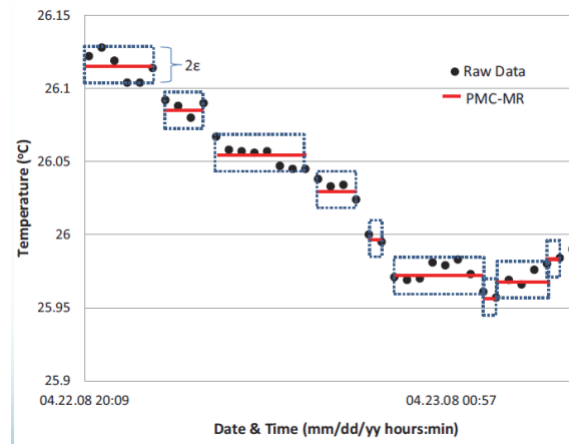


Model-based compression

The last thing of model base that can be used is compression. The goal is to approximate a sensors data stream by a set of functions.

In the example we have a constant value model (very simple) this method is called PMC-MR, The idea is that we start with some value and we consider the value that are

close the the first value and until they are fare away from the constant we just keep this model, if they are away we just start an other constant. So we have several constant in time.



Chapter 9

Context

9.1 Understanding context

Traditional application cannot understand the "context" of a request, then provide data from a request, like a db query. In mobility we may ask this question to the system: "What's the best route to go from X to Y, by car, considering that it's rush hour and that I hate highways?" and also in this question I want to explicit that is rush hour and I don't want to pay the highway. So in traditional applications we must make explicit all the request parameters. In general we would like to do better than this because communicating with a computer is hard:

- Limited interfaces, try to have some automatic mechanism to understand the user needs
- We need automatic mechanisms to acquire and transmit the context of our request

9.2 Defining context

If we understand the context we can just ask "how do I get there" and the system should understand from the context:

- where the user is
- where he wants to go
- how (on foot, car or public transport)

Many definition of context have been proposed for computer science application. By enumeration we can describe some useful context for personalising services:

- location, time, surrounding people, season, temperature
- location, environment, identity and time
- mood, level of attention, location, time, surrounding objects and people

One definition that is used by many that try to define context without enumerating: "Context is any information that can be used to characterize the situation of an entity. An entity is a person, place, or object that is considered relevant to the interaction between a user and an application, including the user and applications themselves". Simplifying context is all the data useful to adapt a service, what characterize the situation when I ask for service, we cannot enumerate all.

We may consider specific categories of context based on:

- U = user
- S = sensors: based on sensors how much reliability of data, which kind of granularity
- D = device: there was a standard in which there was an external repository in which there was information about device that user used. And based on device server-side we can make some changes in providing information
- O = other

Primary dimension	Secondary dimensions	Sources
User	Identity	U
	Physiological	S
	Emotional	S
	Interests	U, O
	Preferences	U, O
	Social	U, S, O
	Organizational	U, O
	Activity	U, S, O
Environment	Weather	S
	Temperature	S
	Humidity	S
	Light	S
	Noise	S
	Air pollution	S
	Surrounding persons / devices	S, O
Location	Physical	S
	Symbolic	S, O
Time	Absolute	D, S
	Granularity-based	D, S, O
Device	Capabilities	D
	Status	D
Connectivity		D, O

9.2.1 Temporal context

Time is not only about time of day, month, season, time zone. We can know something more about current situation if we know some history context. A typically example is in driving context, by knowing a sequence of positions we can obtain a trajectory and observing a recurrent trajectory at a given time we can predict current destination.

9.3 Adaptiveness

Is the property of a system to adapt to a given context for providing a better service/-experience. A context-aware system acquires context data to automatically adapt its behavior.

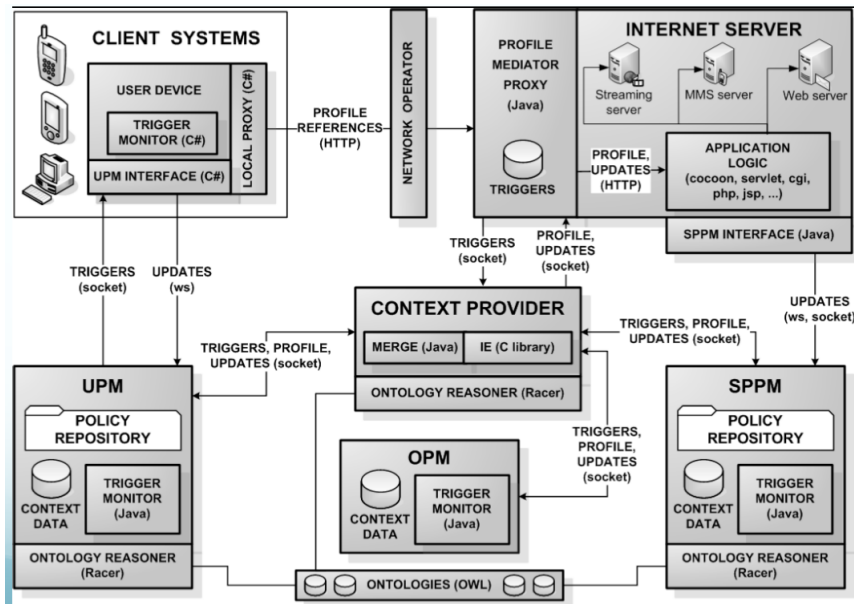
A context-aware system acquires context data to automatically adapt its behavior. It's very important for mobile computing because of:

- changes in network connectivity
- changes in energy availability (battery)
- limited interfaces to specify parameters
- changes in user situation (location, time, activity, ...)
- changes in environment (light, temperature, crowd, ...)

9.3.1 Adaptive video streaming

We don't have to analyze this, the idea is that the bottom is a module that is trying to get context and the application will use the right streaming based on the context, for example if there is a bad connectivity. The parameters are the capability of the network system and the network bandwidth that can change quickly. They were dynamically

degrade the quality while decreasing the bandwidth in order to not have stuttering. Based on estimated network bandwidth and device battery level, video quality is continuously changed without interrupting the service.



9.3.2 Type of adaptiveness

There are two different ways to adapt:

- Adapting functionality:
 - The system may change the data flow, hide or expose functionalities or decrease service precision. Change functionality of the system in order to preserve the service.
 - E.g: increase caching, move more computation server-side, change interface mode, hide security sensitive functionalities (for example the monitor in hospital turn off if some people that haven't the grant to see that information are in the room)
- Adapting data:
 - The system may provide less or more precise position data, use higher or lower quality images, change the sampling rate from sensors

9.4 Obtaining context data

A low level context are data acquired from sensors or other sources, example: raw data from sensors, explicit user preferences from profiles, mobile device capabilities. But we consider low level even if we do some simple processing and/or fusion of raw data, definition of simple is not trivial. Simple for example is calculate the average. For example:

- Taking bandwidth estimation by averaging over a set of samples in a time window
- Assuming "Hot&Humid" if average of temp values from different sources in the area are over 30°C and humidity is over 80%

9.5 Inferring context data

Hig-level context is when we apply some inference methods on low level context:

- the activity "cooking a meal" inferred by domotics sensors data and positioning, by a machine learning algorithm. Human Activity is considered hig-level context, a gesture is an activity? no, activity is a sequence of action, maybe even parallel action. But we consider gesture hig-level context because we have to involve some process, for example to distinguish different gesture (for example machine learning).
- User's mood derived by combining data from the activities performed in the day, GSR sensed at his wrist, and face recognition if available. Again, even only the GSR in order to go from the signal to actually the mood they use machine learning algorithm and clearly it's high level context.

Inference can also be obtained by reasoning on high-level context and common knowledge. Common sense is something that we know from our experience (for example we know that in a kitchen there is a fridge, not because we've seen it but because we know from our experience of kitchen users).

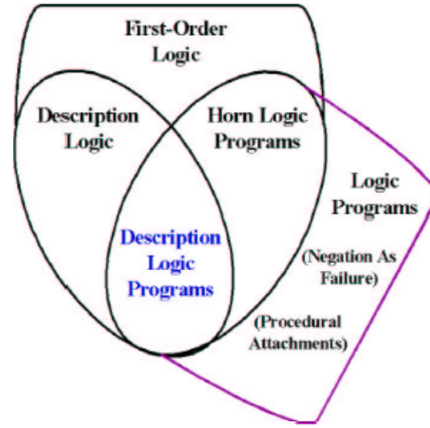
9.5.1 Inference methods

This could be an other question. How do we capture high-level context, with inference methods.

Are two approaches:

- Symbolic approaches:
 - We can reason in an efficient way on common sense
 - Horn logic programs
 - General logic programs
 - Decription logics
 - Description logic plus rules
- Statistical approaches (mostly supervised machine learning):
 - Classification
 - Clustering
 - LLM

both of them could be usefully for reasoning about context. There are also hybrid approaches, are called neuro-symbolic.

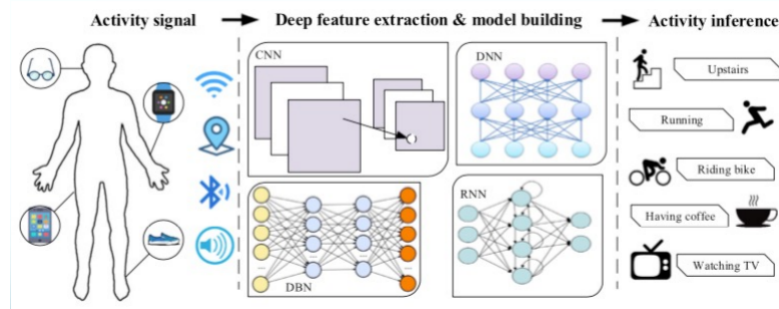


9.6 Human activities

There are libraries that acquire data from sensors in mobile phone, but they are very simple and imprecise. So we have to use a different approach to obtain human activities context, for example using deep learning.

9.6.1 Deep Learning approach

We get data from sensors in devices (for ex. acceleration, position and so on). We also get data from environment and then we have to pre-process this data (data cleaning, applying windowing and so on) and also in the traditional models we need to collect ground truth because we'll use supervised machine learning model and we need the real situation of user in order to validate the model. After the data cleaning process we go into deep learning model, in which we have automatic feature extraction, the connection of deep learning model have weight and the weight are tuned with the ground truth. After that the model are ready and we can deploy the model on the user's device. The system use what has learned in the past to understand if the pattern in the window are similar to something that it knows.

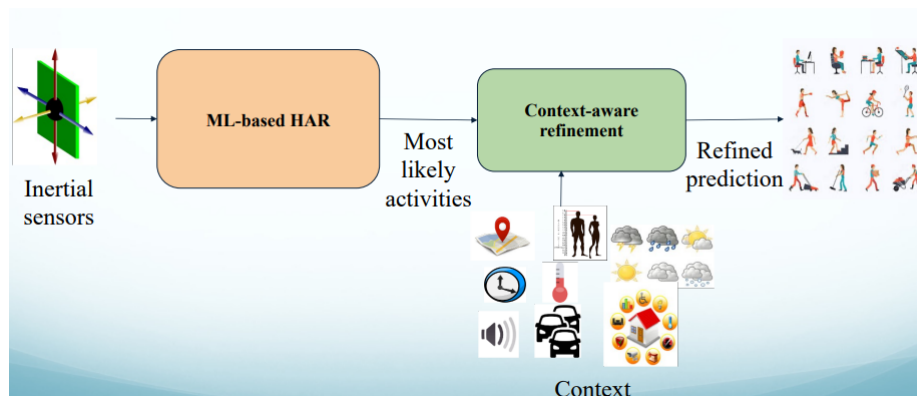


9.6.2 Issues with supervised ML/DL approaches

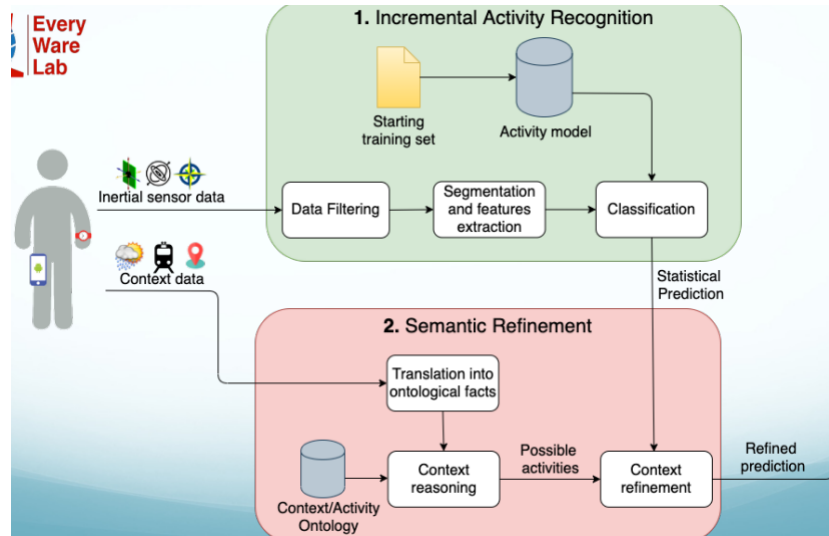
There are a lot of problems with this kind of approach. This thing about observing people in order to build the model (collect the ground truth) is expensive and sometimes is impossible, what about activity that we do at home and in private space? we can set up a fake smart home in the lab and then we have actors that show how they do things. But it's very difficult that this kind of training works in real life situation. An other problem for collecting data is the high variability of environments, situations, sensors and infrastructure setup.

9.6.3 Hybrid data-driven and KB solution

They are trying to use context. They pass the data collect from inertial sensor through a ML-based HAR and then they do a context-aware refinement from common sense or a high-level context. For example if outside is raining some activities are less probable than usual. They did this because the system give the same output if the user was brushing teeth and writing on a blackboard, so they used GNSS to understand the position (if the position was in university it's most probable that the user was writing on a blackboard than that it was brushing teeth).



The problem is having a lot of labeled data in HAR, so there are a lot of research on unsupervised model for HAR, that are model that don't have labels. Or semi-supervised model, for example active learning, when system is unsure ask to the user, we use the feedback from the user as a ground truth. In the figure we see an hybrid model that use statistical model and context. Incremental because we want to improve the model to get better and better, it means that we have to train again when we can do some improvement.



9.7 Context representation

What we can use to represent context? we may use very simple representation just storing key-value pairs (temperature this is the value) using simple file or noSQL database. Or we can use some representation that capture the relationship with context value. The representation give us also inference ability.

Having context representation is important for different reasons. The main motivation to have a formal representation of the data is for automatically process them. Also if we have context it could be used for several services, so having a formal representation is useful, in order to do that we need a common representation language.

In some sense we would like to build middelwere, like in smartphone in which we can use some API to access context data. Context model should support:

- Heterogeneity of context data
- Relationships among context data
- Context history
- Uncertainty (including imprecision) of context data
- Reasoning on context data
- Usability
- Efficiency

9.7.1 Flat Model

No structure and generally tailored to single applications. It's based on key/value pairs that are used by commercial applications servers.

We can also have some markup language in order to give a little bit of structure of data, like XML or JSON. We can obtain hierarchies of attributes. Are supported by well-established technologies useful for marshalling and unmarshaling.

One of the issue with all representation is for the same context data different people use different words.

```

1 {
2   "provider_id": 2216758877,
3   "provider_type": "smartphone",
4   "provider_name": "phone01",
5   "time" : [11, 31],
6   "period" : "PM",
7   "date" : [12, 19, 2018],
8   "latitude": [33, 59, 14, 3],
9   "longitude": [-118, 13, 32, 0],
10  "city" : "Los Angeles",
11  "zip" : 90089,
12  "event" : "heart attack",
13  "domain": "healthcare",
14  "name" : "Paul Rodriguez"
15 }

```

The problems in request to requirements:

- Expression of relations and dependencies
- Support for historical data
- Support for uncertainty
- Reasoning

	Flat	DB based	Ontological
Heterogeneity and extensibility	+/-		
Expression of relations and dependencies	-		
Support for historical data	-		
Support for uncertainty	-		
Reasoning	-		
Usability	+		
Efficiency	+		

9.7.2 DB-based models

In DB we can represent relationship in form of tables. We are facilitate modeling by using abstraction like classes, properties, relations. We have tools like DBMS. We don't have reasoning support in Db, not even consistency checking.

The main thing we gain is the expression of relations and dependencies.

	Flat	DB-based	Ontological
Heterogeneity and extensibility	+/-	+/-	
Expression of relations and dependencies	-	+	
Support for historical data	-	-	
Support for uncertainty	-	+/-	
Reasoning	-	-	
Usability	+	+	
Efficiency	+	+	

9.7.3 Ontological models

In computer science, by ontology we mean: a formal (logic) specification of a shared conceptualization. Semantic search is retrieving other data that are related to the answer of user. The domain of interest is described in terms of classes, relations, individuals and properties. The used language is description logics:

- family of first-order logic subsets
- they allow modeling very complex domains
- formal semantics
- automatic consistency checking and reasoning

Like what happens in semantic web, behind the semantic web there is description logic.

OWL2 is the language for defining ontologies. Complex descriptions can be obtained by composing simpler ones:

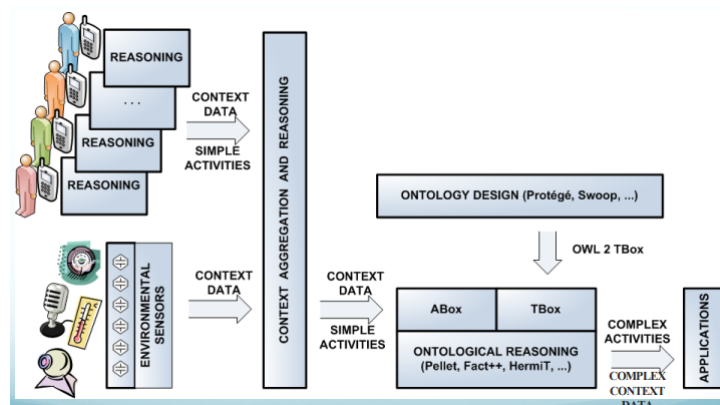
$\text{CarnivalParty} \sqsubseteq \text{FriendlyMeeting} \sqcap \forall \text{hasActor}.(\text{Person} \sqcap \exists \text{isWearing.Mask.})$
$\text{FriendlyMeeting} \sqsubseteq \text{Activity} \sqcap \geq 2 \text{hasActor.Friend.}$

Carnival party is a friendly meeting and people are participating in person and wearing mask. We can go on and define what is friendly meeting, at least there are 2 people and they have to be friends. We can see another, more complex, example:


```

TeaParty  $\sqsubseteq$  SocialActivity  $\sqcap \forall$ hasTimeExtent.Afternoon  $\sqcap$ 
 $\forall$ hasActor.(Person  $\sqcap \exists$ hasCurrentPosture.Seated
 $\sqcap \exists$ hasCurrentActivity.Sipping  $\sqcap \exists$ hasCurrentLocation.
(LivingRoom  $\sqcap \geq 2$  contains.TeaCup
 $\sqcap \forall$ hasSoundSensor.(MeasuredSoundDb  $\leq 35$ [int]))),

```



	Flat	DB-based	Ontological
Heterogeneity and extensibility	+ / -	+ / -	+
Expression of relations and dependencies	-	+	+
Support for historical data	-	-	-
Support for uncertainty	-	+ / -	+ / -
Reasoning	-	-	+
Usability	+	+	+
Efficiency	+	+	-

9.8 Uncertainty

Using logic is a problem related to uncertainty. Our perception of the world, as well as measurements we take, are prone to uncertainty:

- "it's hot" / "it's cold"
- Different sensors may provide different values for the same data

It's important to manage uncertainty and conflicts among context data.

In the past we tried activity recognition only based on this patter, was designed by activity engineer.

Support for automatic reasoning:

- subsumption (is this concept a specialization of this other concept?)
- realization (which concept captures this set of observations?) consistency (is there any contradiction in our definitions of concepts?)

9.8.1 Handling Uncertainty

Incompleteness:

- The data we need is missing!
- Difficult to handle; do not use the value, or use default values instead

Inaccuracy:

- The data is imprecise
- Data aggregation to improve precision (if multiple sources are available)
- Probabilistic logics "if the probability that the user has fallen is more than 30%, call the medical center"

Temporal uncertainty, in order to predict an activity is useful to know what happens before, with the past of time we may want to say to give more importance to the recent data. But we can have some problems:

- "Old" data may not be valid now
- Use a time-to-live
- Data prediction
- Progressively lower the confidence in the value

Deal with conflicts, different sources provide contradictory values for the same data, the user is in two different places at the same time. Choose the most reliable value (and/or consider majority). And we also need to deal with inconsistency, some data are not compatible with others, the user is in a car and he is walking at the same time. Use automatic consistency check tools.

Nowadays there is no standard. But as we said we can use different approaches:

- Key/Value models:
 - They support uncertainty only for particular data (e.g., location)
- CML:
 - Quality metadata (timestamp, accuracy, probability, ...)
 - Handle inconsistencies through three-valued logic (true, possibly true, false)
- Ontological models:
 - Automatic consistency checking
 - Some languages support probability and fuzziness, but sometimes they are less expressive than OWL 2

9.8.2 Context Management Platform

A software layer below application that offers common context services. Which are the features:

- Context acquisition from different sources
- Context fusion and conflict resolution
- Context reasoning
- Privacy protection

It's not mentioned the context representation, but it's clear that we have to represent it.

Sometimes we confuse all the stuff that we did in sensor data management is different in respect to what we did here. We've seen basic data acquisition methods, then we introduce model-based (mainly to save energy). So "which are the main methods to derive high-level context?" are related to what we did today. Typical question: give an example of high level context and low level context and how you would use them in an application.